

Decision Tree

Yongmei Tan

ymtan@bupt.edu.cn

小故事：决策树与罗斯·昆兰

说起决策树学习，就必然要谈到澳大利亚计算机科学家罗斯·昆兰 (J. Ross Quinlan, 1943—)。

最初的决策树算法是心理学家兼计算机科学家 E. B. Hunt 1962 年在研究人类的概念学习过程时提出的 CLS (Concept Learning System)，这个算法确立了决策树“分而治之”的学习策略。罗斯·昆兰在 Hunt 的指导下于 1968 年在美国华盛顿大学获得计算机博士学位，然后到悉尼大学任教。1978 年他在学术假时到斯坦福大学访问，选修了图灵的助手 D. Michie 开设的一门研究生课程。课上有一个大作业，要求写程序来学习出完备正确的规则，以判断国际象棋残局中一方是否会在两步棋后被将死。昆兰写了一个类似于 CLS 的程序来完成作业，其中最重要的改进是引入了信息增益准则。后来他把这个工作整理出来在 1979 年发表，这就是 ID3 算法。



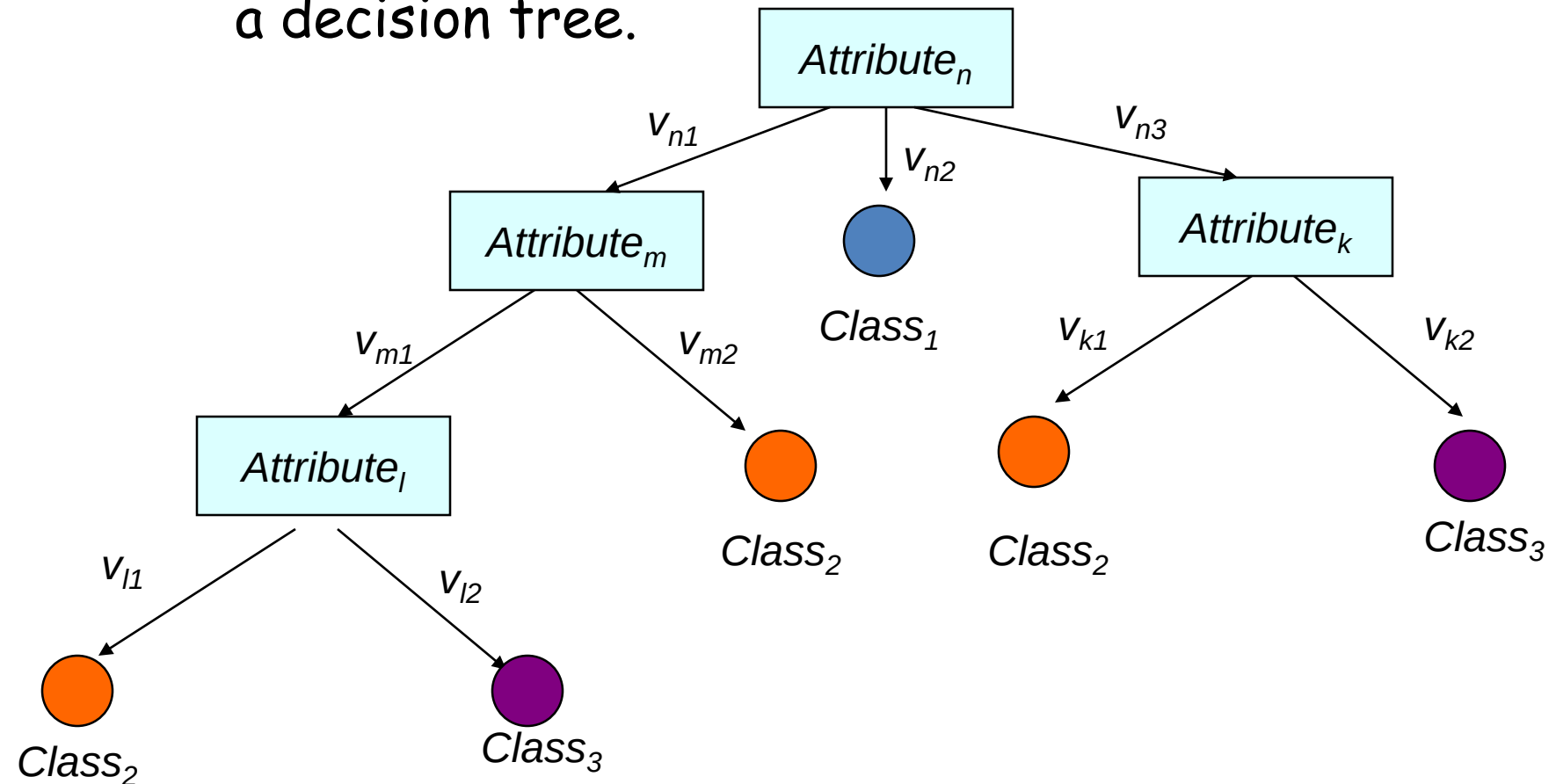
1986 年 *Machine Learning* 杂志创刊，昆兰应邀在创刊号上重新发表了 ID3 算法，掀起了决策树研究的热潮。短短几年间众多决策树算法问世，ID4、ID5 等名字迅速被其他研究者提出的算法占用，昆兰只好将自己的 ID3 后继算法命名为 C4.0，在此基础上进一步提出了著名的 C4.5。有趣的是，昆兰自称 C4.5 仅是对 C4.0 做了些小改进，因此将它命名为“第 4.5 代分类器”，而将后续的商业化版本称为 C5.0。

Decision Tree

WHAT IS A DECISION TREE

Decision Trees

A method for approximating discrete-valued target functions. The learned function is represented by a decision tree.



Decision Trees and if-then Rules

Idea: ask a series of questions about the attributes of an instance in order to arrive at the correct classification.

输入: 训练集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$;

属性集 $A = \{a_1, a_2, \dots, a_d\}$.

过程: 函数 TreeGenerate(D, A)

1: 生成结点 node;

2: **if** D 中样本全属于同一类别 C **then**

3: 将 node 标记为 C 类叶结点; **return**

4: **end if**

5: **if** $A = \emptyset$ **OR** D 中样本在 A 上取值相同 **then**

6: 将 node 标记为叶结点, 其类别标记为 D 中样本数最多的类; **return**

7: **end if**

8: 从 A 中选择最优划分属性 a_* ;

9: **for** a_* 的每一个值 a_*^v **do**

10: 为 node 生成一个分支; 令 D_v 表示 D 中在 a_* 上取值为 a_*^v 的样本子集;

11: **if** D_v 为空 **then**

12: 将分支结点标记为叶结点, 其类别标记为 D 中样本最多的类; **return**

13: **else**

14: 以 TreeGenerate($D_v, A \setminus \{a_*\}$) 为分支结点

15: **end if**

16: **end for**

输出: 以 node 为根结点的一棵决策树

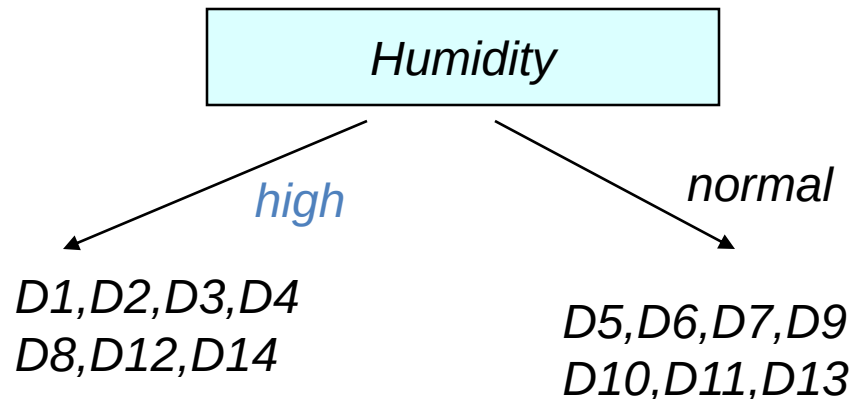
决策树基本学习算法

Decision Trees

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Decision Trees

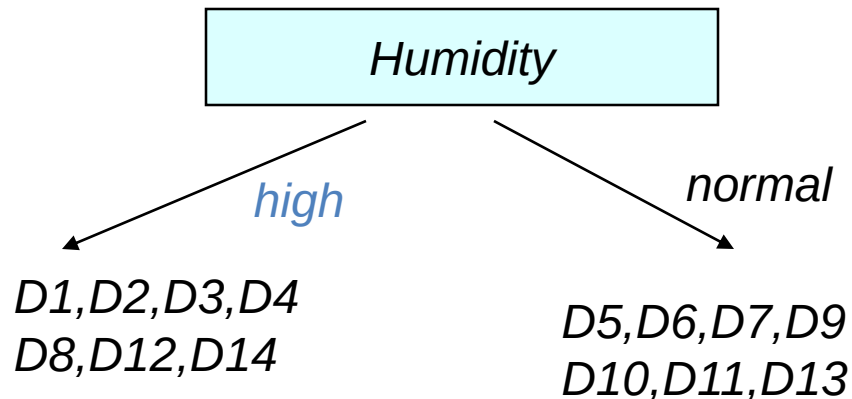
Lets start building the tree from scratch. We first need to decide which attribute to make a decision. Lets say we selected **“humidity”**



Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Decision Trees

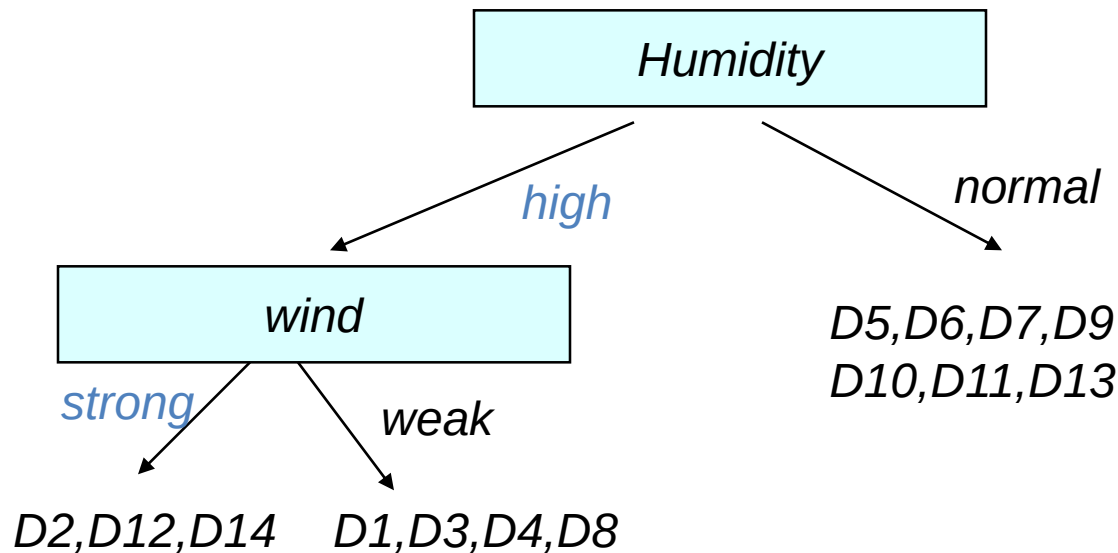
Now lets classify the first subset
 $D1, D2, D3, D4, D8, D12, D14$ using attribute
“wind”



Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D8	Sunny	Mild	High	Weak	No
D12	Overcast	Mild	High	Strong	Yes
D14	Rain	Mild	High	Strong	No

Decision Trees

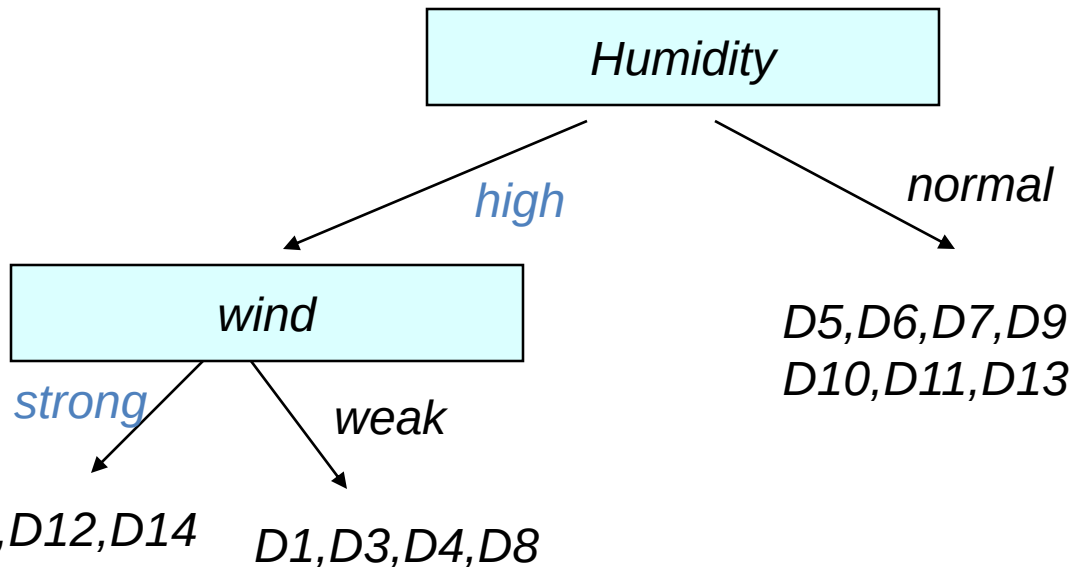
Subset $D1, D2, D3, D4, D8, D12, D14$ classified by attribute “**wind**”



Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D8	Sunny	Mild	High	Weak	No
D12	Overcast	Mild	High	Strong	Yes
D14	Rain	Mild	High	Strong	No

Decision Trees

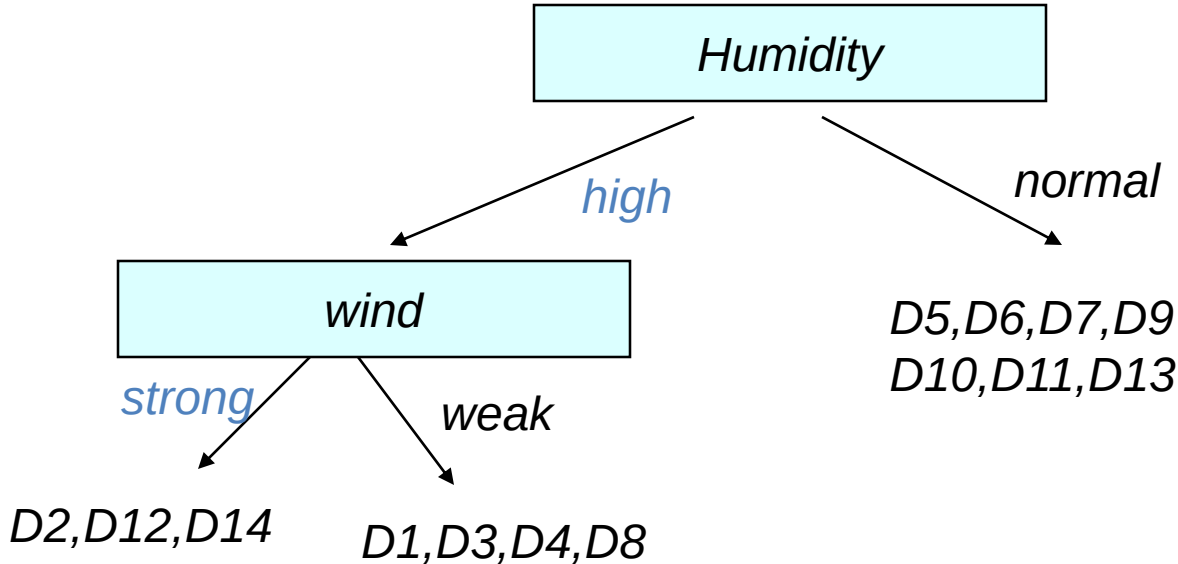
Now let's classify the subset $D2, D12, D14$ using attribute "**outlook**"



Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D2	Sunny	Hot	High	Strong	No
D12	Overcast	Mild	High	Strong	Yes
D14	Rain	Mild	High	Strong	No

Decision Trees

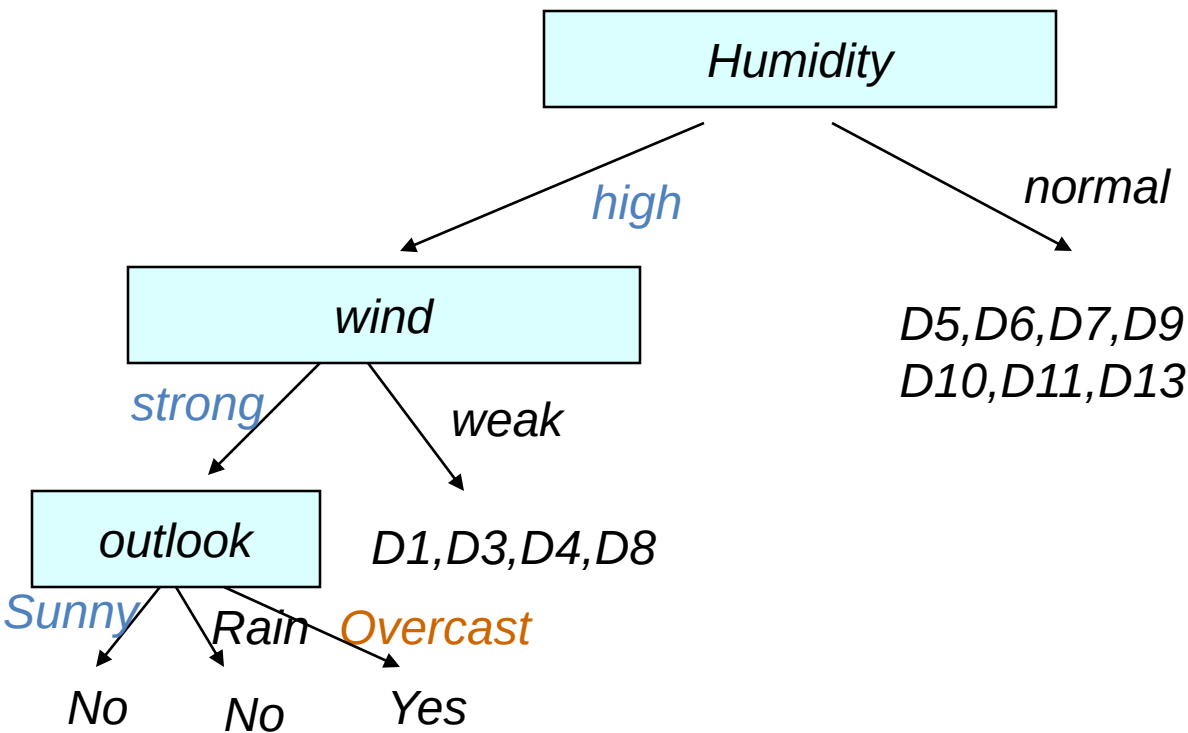
Subset $D2, D12, D14$ classified by “**outlook**”



Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D2	Sunny	Hot	High	Strong	No
D12	Overcast	Mild	High	Strong	Yes
D14	Rain	Mild	High	Strong	No

Decision Trees

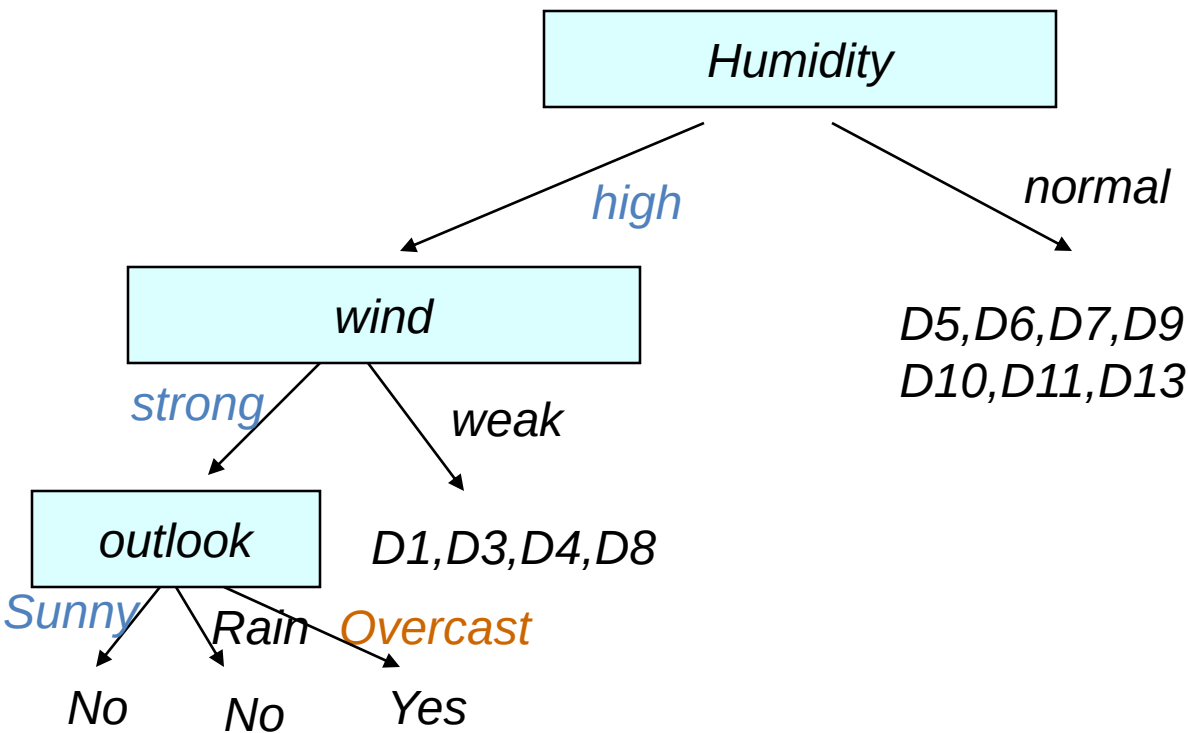
subset $D2, D12, D14$ classified using attribute **“outlook”**



Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D2	Sunny	Hot	High	Strong	No
D12	Overcast	Mild	High	Strong	Yes
D14	Rain	Mild	High	Strong	No

Decision Trees

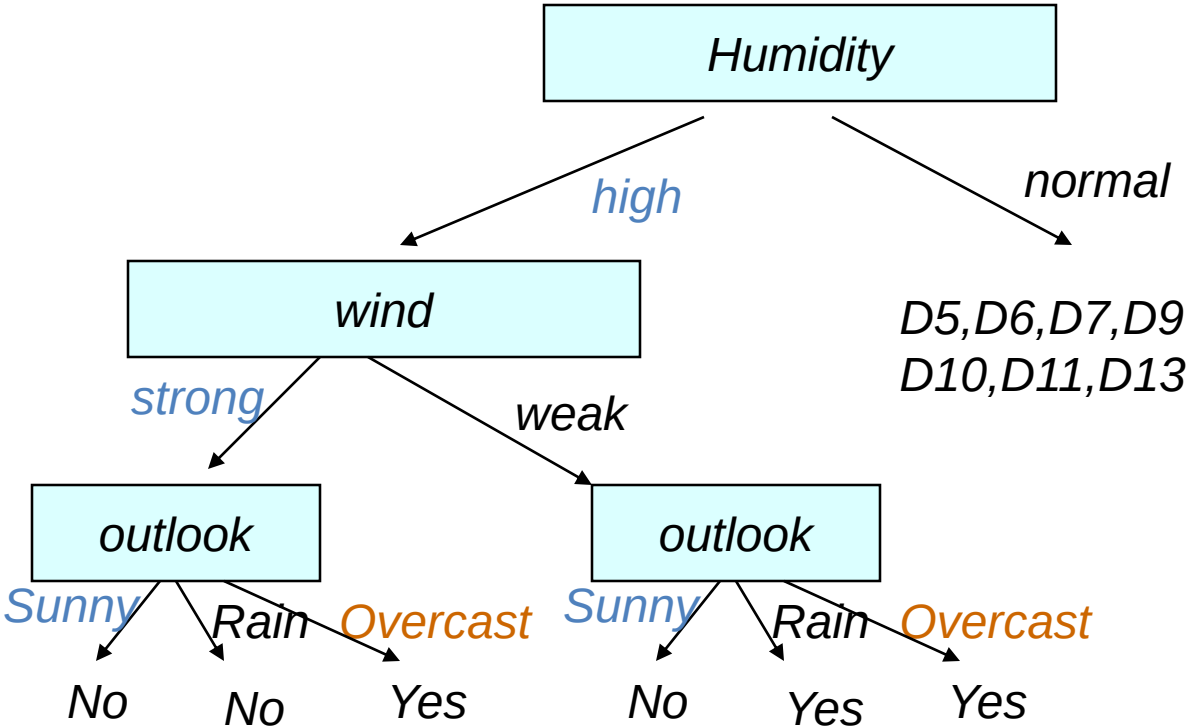
Now lets classify the subset $D1, D3, D4, D8$ using attribute “**outlook**”



Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D8	Sunny	Mild	High	Weak	No

Decision Trees

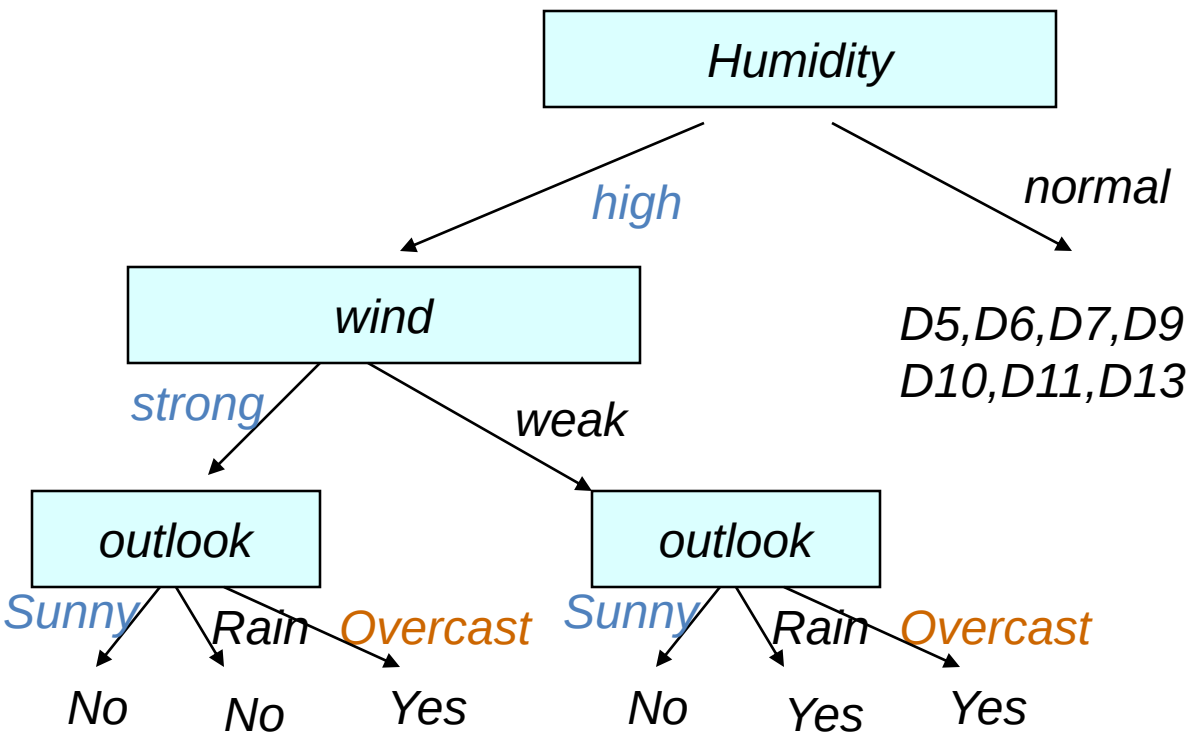
subset *D1,D3,D4,D8* classified by “**outlook**”



Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D8	Sunny	Mild	High	Weak	No

Decision Trees

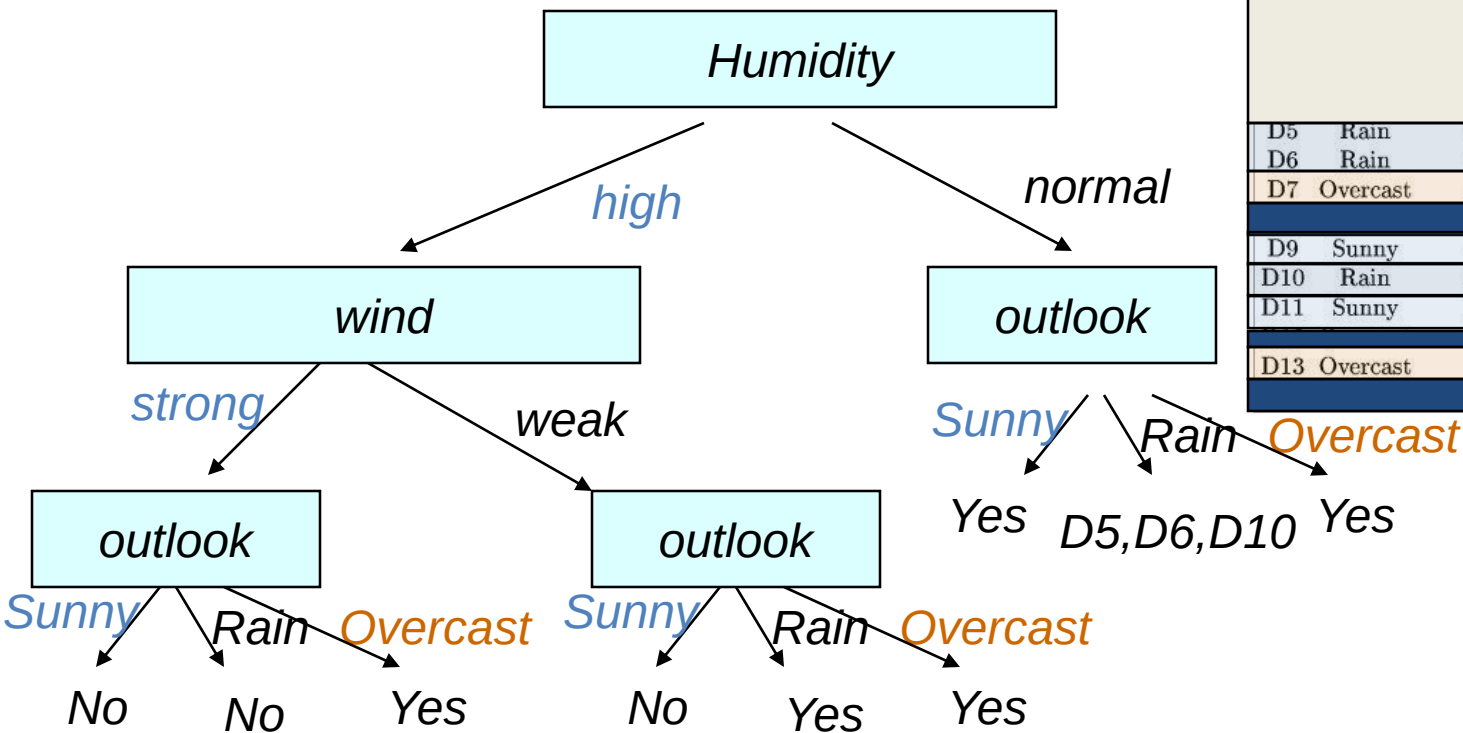
Now classify the subset
D5,D6,D7,D9,D10,D11,D13 using attribute
“outlook”



Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes

Decision Trees

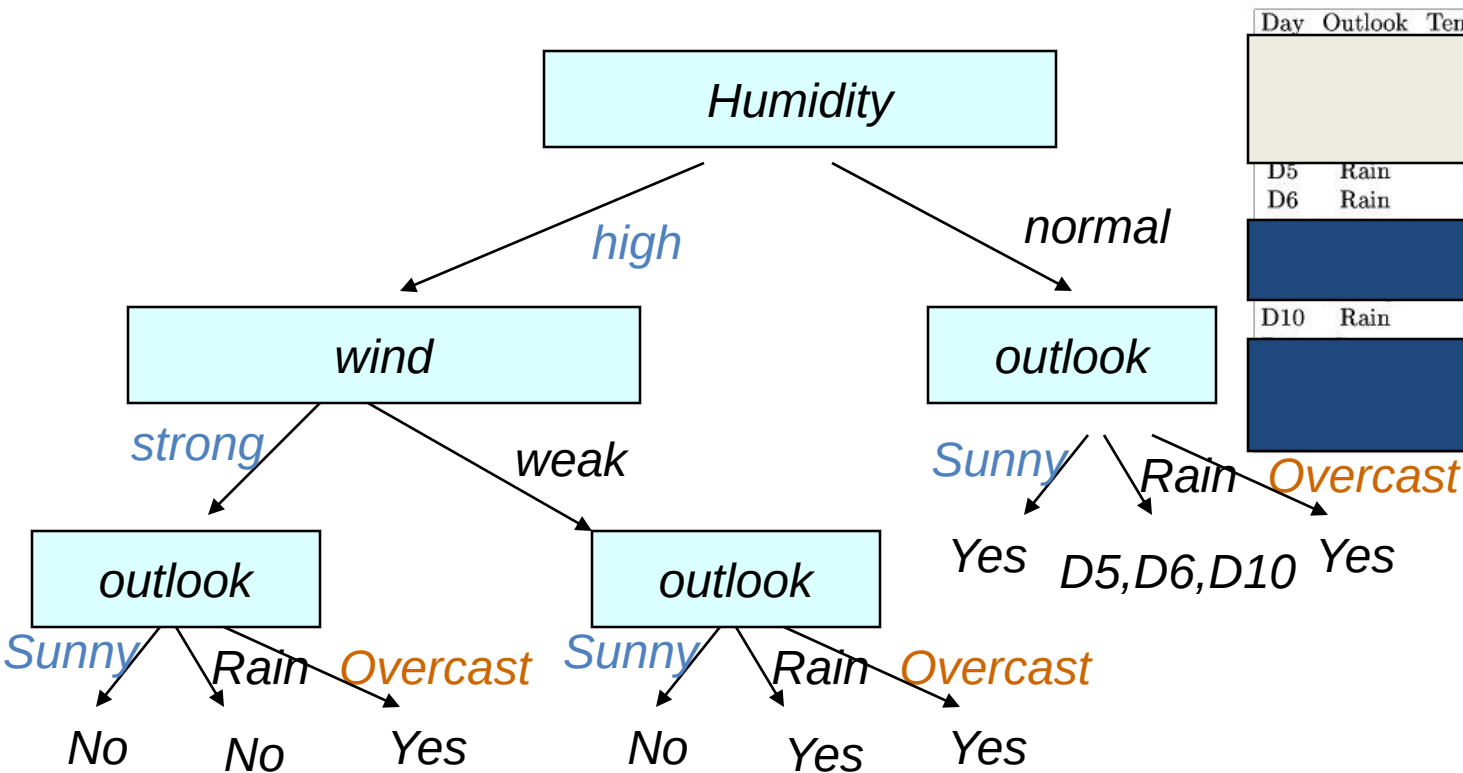
subset $D5, D6, D7, D9, D10, D11, D13$ classified by
“outlook”



Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes

Decision Trees

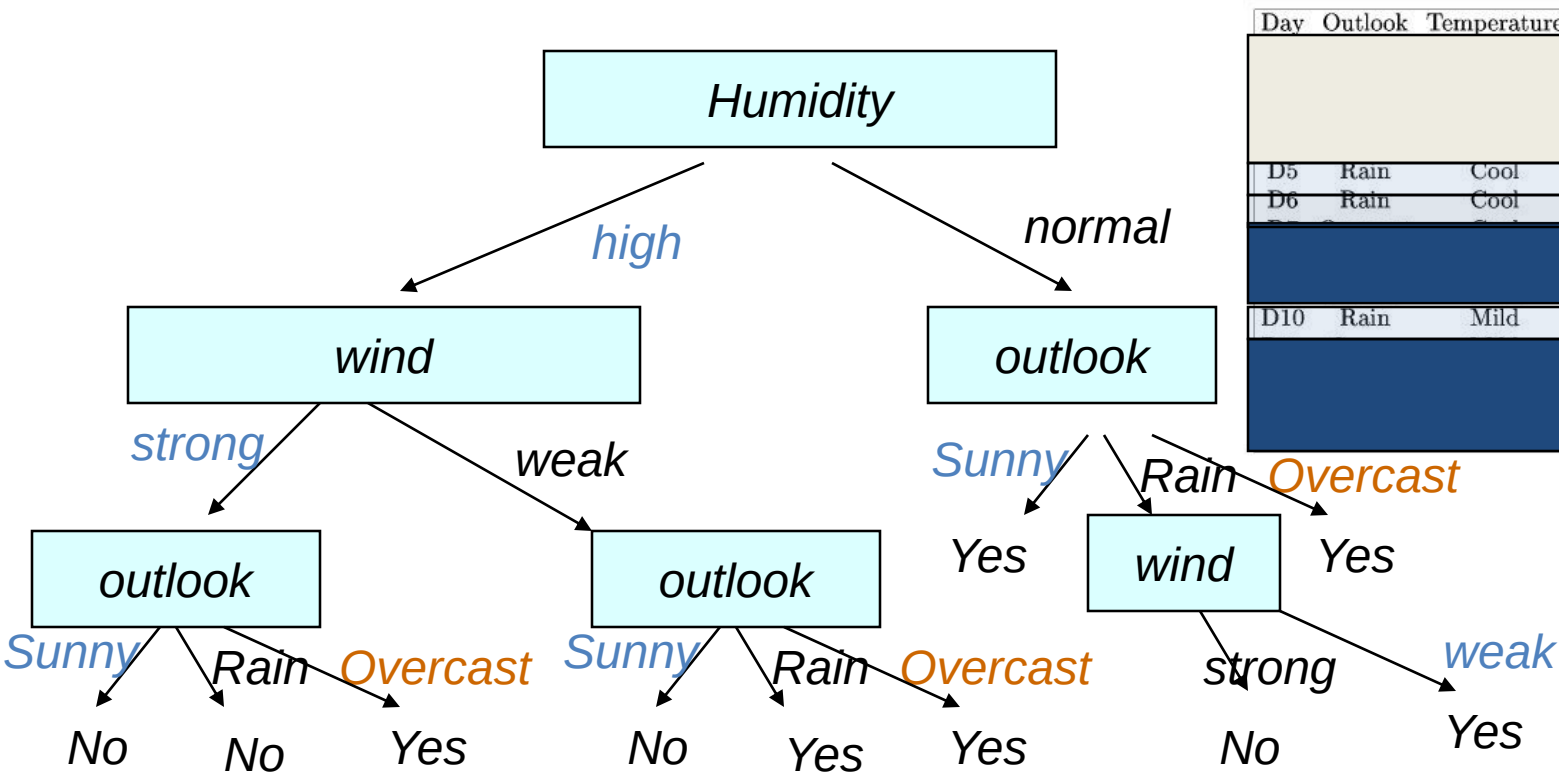
Finally classify subset D5,D6,D10 by “*wind*”



Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D10	Rain	Mild	Normal	Weak	Yes

Decision Trees

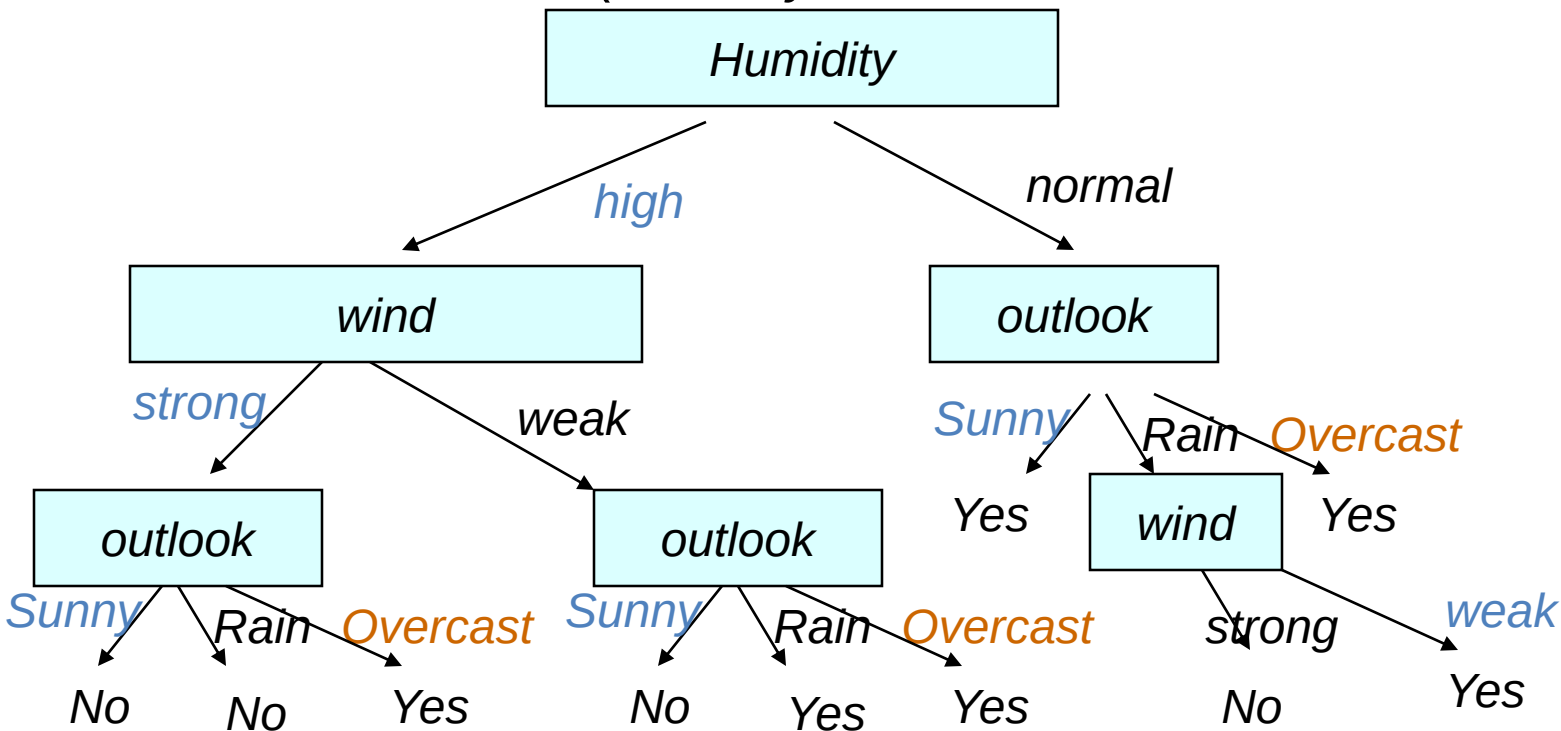
subset D5,D6,D10 classified by “wind”



Decision Trees

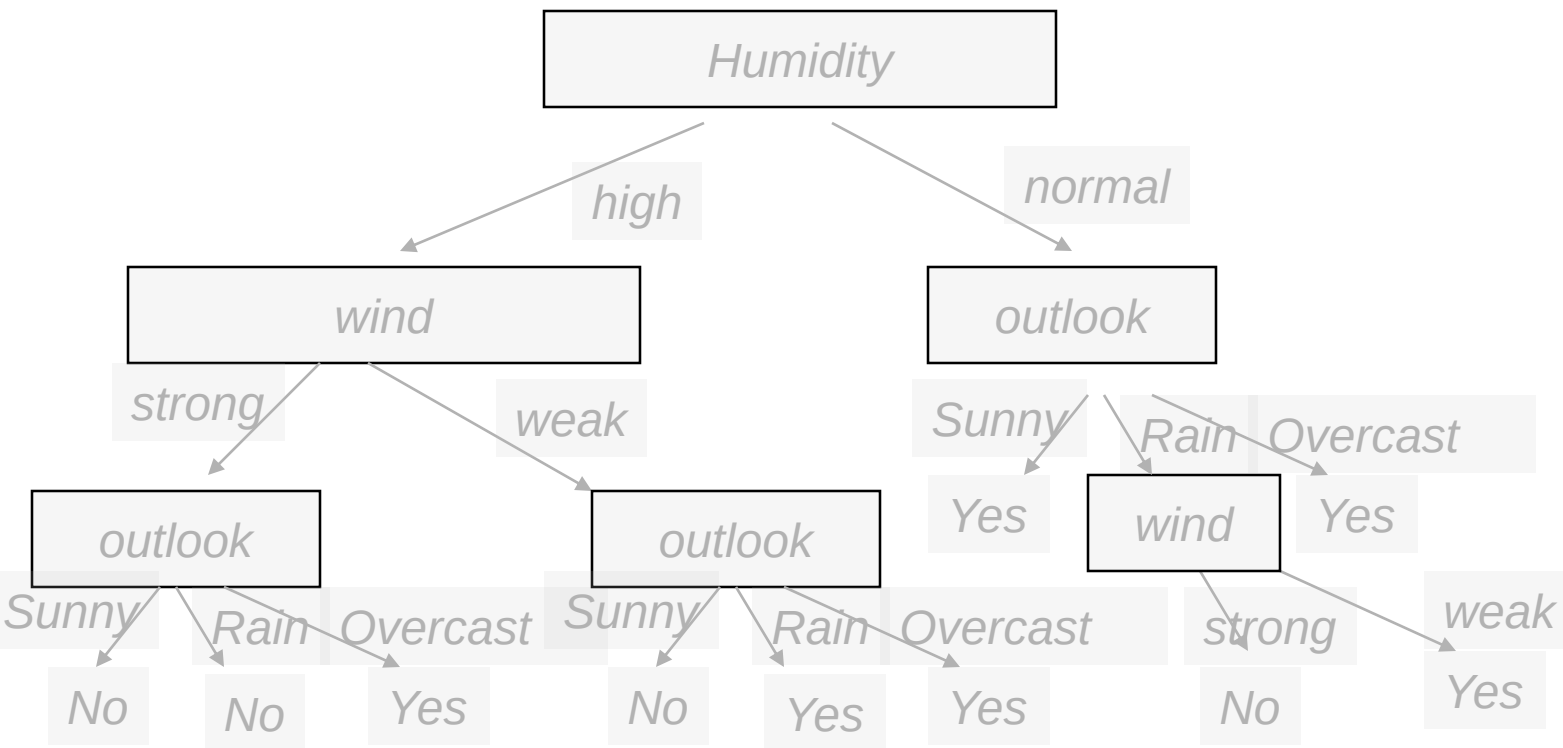
Note: The decision tree can be expressed as an expression or if-then-else sentences

$(\text{humidity}=\text{high} \cap \text{wind}=\text{strong} \cap \text{outlook}=\text{overcast}) \cup$
 $(\text{humidity}=\text{high} \cap \text{wind}=\text{weak} \cap \text{outlook}=\text{rain}) \cup$
 $(\text{humidity}=\text{high} \cap \text{wind}=\text{weak} \cap \text{outlook}=\text{overcast}) \cup$
 $(\text{humidity}=\text{normal} \cap \text{outlook}=\text{sunny}) \cup$
 $(\text{humidity}=\text{normal} \cap \text{outlook}=\text{overcast}) \cup$
 $(\text{humidity}=\text{normal} \cap \text{outlook}=\text{overcast} \cap \text{wind}=\text{weak})$



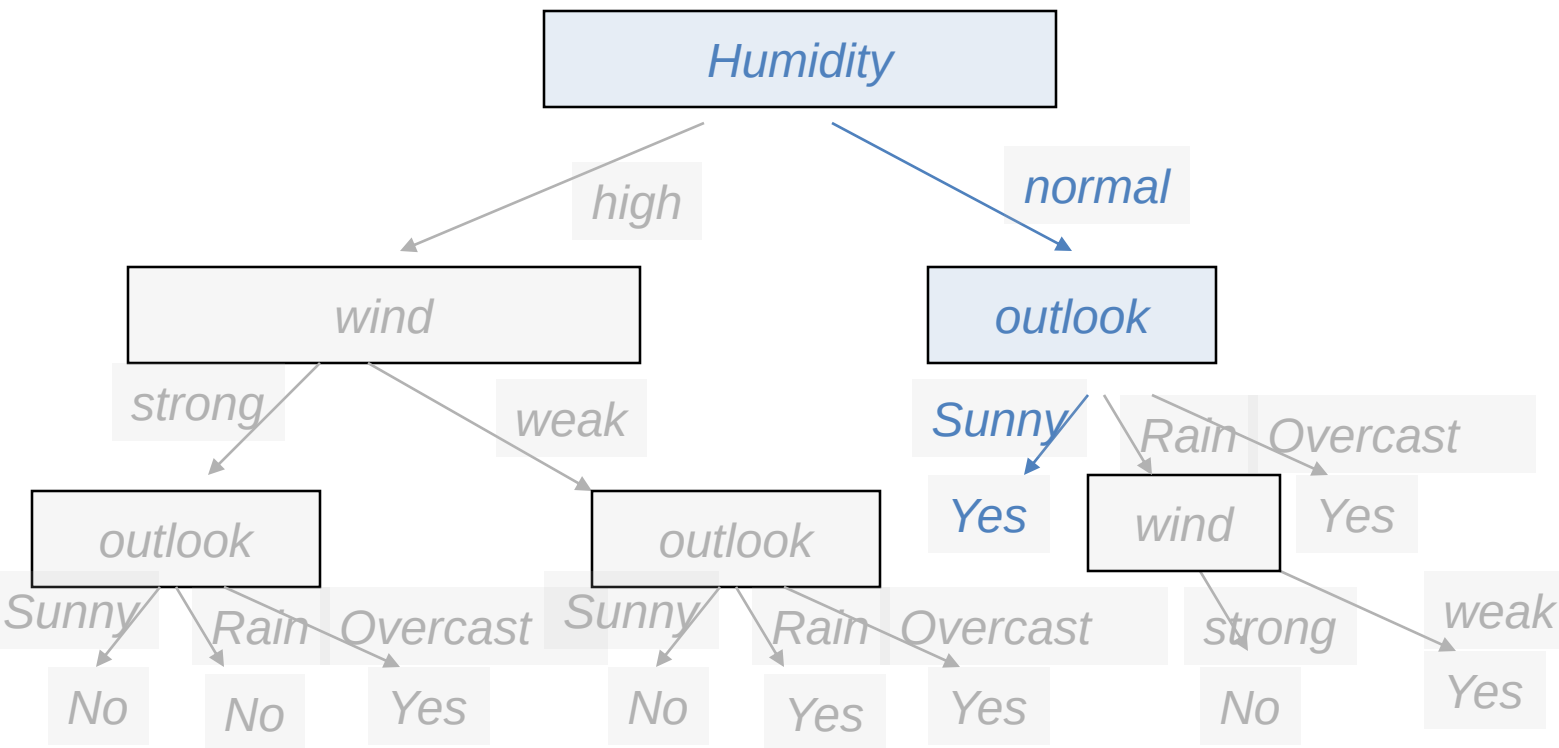
Decision Trees

Now classify $\langle \text{sunny}, \text{hot}, \text{normal}, \text{weak} \rangle = ?$



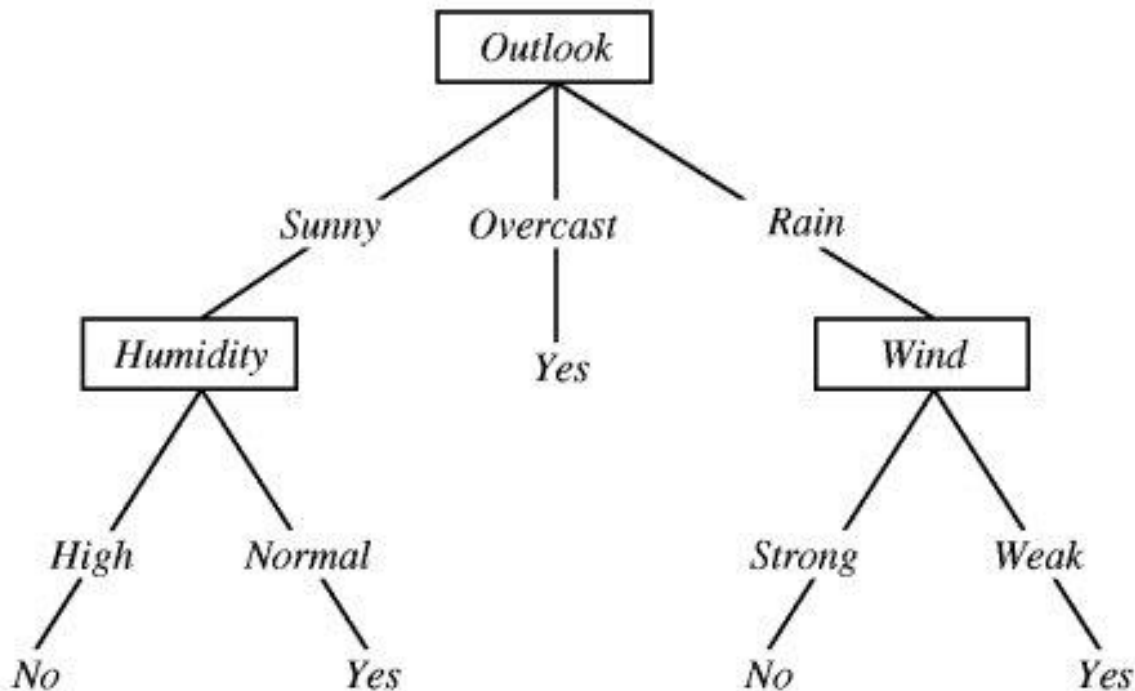
Decision Trees

Now classify $\langle \text{sunny}, \text{hot}, \text{normal}, \text{weak} \rangle = ?$



Decision Trees

Another tree from the same training data using different attributes



Question: What is the maximal number of (unique) decision trees we can possibly build?

Which attribute we should choose at branches?

Decision Tree

ID3

Entropy

A measure of the *disorder or randomness* in a *closed system*

In our system we have 9 positive examples and 5 negative examples:
 $Entropy(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$

$p_+ = 9/14$ (ratio of positive examples over all examples)

$p_- = 5/14$ (ratio of negative examples over all examples)

So entropy of our system is

$$Entropy(S) = -(9/14) \log_2(9/14) - (5/14) \log_2(5/14) = 0.940$$

Note that in a homogenous system
The entropy will be 0 (why?) and if
both classes has equal numbers of
instances then the entropy will be 1.

Entropy

If there are more than two classes:

$$\text{Entropy}(S) = \sum_{i=1}^n -p(i) \log_2 p(i)$$

Where n is the number of classes

Entropy([9+,5-])

$= -(9/14)\log_2(9/14) - (5/14)\log_2(5/14)$

$= 0.940$

Conditional Entropy

Given attribute X , the entropy of the system.

*We can evaluate each attribute by calculating how much change they will do in entropy. For example, we can evaluate the attribute “**Temperature**”.*

There are 3 values (Hot,Mild,Cool). So we can create 3 subsets for each value.

$$S_{hot}=\{D1,D2,D3,D13\}$$

$$S_{mild}=\{D4,D8,D10,D11,D12,D14\}$$

$$S_{cool}=\{D5,D6,D7,D9\}$$

Lets find entropy in each subset

$$Entropy(S_{hot}), Entropy(S_{mild}), Entropy(S_{cool})$$

$S_{hot}=\{D1(-),D2(-),D3(+),D13(+)\}$ (2 positive and 2 negative examples)

$$p_+=0.5$$

$$p_-=0.5$$

$$\text{Entropy}(S_{hot}),=-0.5\log_2 0.5-0.5\log_2 0.5=1$$

$S_{mild}=\{D4 (+),D8(-),D10(+),D11(+),D12(+),D14(-)\}$

$$p_+=0.666$$

$$p_-=0.333$$

$$\text{Entropy}(S_{mild}),=-0.666\log_2 0.666-0.333\log_2 0.333=0.918$$

$S_{cool}=\{D5(+),D6(-),D7(+),D9(+)\}$

$$p_+=.75$$

$$p_-=0.25$$

$$\text{Entropy}(S_{cool}),=-0.25\log_2 0.25-0.75\log_2 0.75=0.811$$

Conditional Entropy

We can represent the entropy of the system when we use attribute “Temperature” as

$$(|S_{hot}|/S) \text{Entropy}(S_{hot}) + (|S_{mild}|/S) \text{Entropy}(S_{mild}) + (|S_{cool}|/S) \text{Entropy}(S_{cool})$$

$$(4/14) + (6/14) * 0.918 + (4/14) * 0.811 = 0.9108$$

Information Gain

We define Information Gain as the difference between the entropy of the system before the split and the expected entropy of the system after the split.

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

$$\text{Gain}(S, \text{Temperature}) = 0.940 - 0.9108 = 0.029$$

Information Gain

- $\text{Gain}(S, \text{Outlook}) = 0.246$ <--- chosen attribute
- $\text{Gain}(S, \text{Humidity}) = 0.151$
- $\text{Gain}(S, \text{Wind}) = 0.048$
- $\text{Gain}(s, \text{Temp}) = 0.029$

*Is information-gain always a positive value?
Why or why not?*

Algorithm

Building a decision tree with ID3 algorithm

- 1. Start from an empty node*
- 2. Select the attribute with the most information gain*
- 3. Create the subsets for each value of the attribute*
- 4. For each subset
if not all the elements of the subset belongs to same class repeat the steps 2-3 for the subset*

Principles

- 1. Simple-to-complex*
- 2. Hill climbing*

Decision Tree Algorithm

• DTL(Examples, TargetAttribute, Attributes)

create a *Root* node for the tree

• If all *Examples* are positive, **return** the single-node tree *Root*, with label = Yes

• If all *Examples* are negative, **return** the single-node tree *Root*, with label = No

• If *Attributes* is empty,

• **return** the single-node tree *Root*, with label = most common value of *TargetAttributes* in *Examples*

• **Else begin**

• $A \leftarrow$ the attribute from *Attributes* with the highest information gain with respect to *Examples*

• Make *A* the decision attribute for *Root*

• **For** each possible value v of *A*

• add a new tree branch below *Root*, corresponding to the test $A = v$

• let $Examples_v$ be the subset of *Examples* that have value v for attribute *A*

• If $Examples_v$ is empty **Then**

• add a leaf node below this new branch with label = most common value of *TargetAttribute* in *Examples*

• **Else**

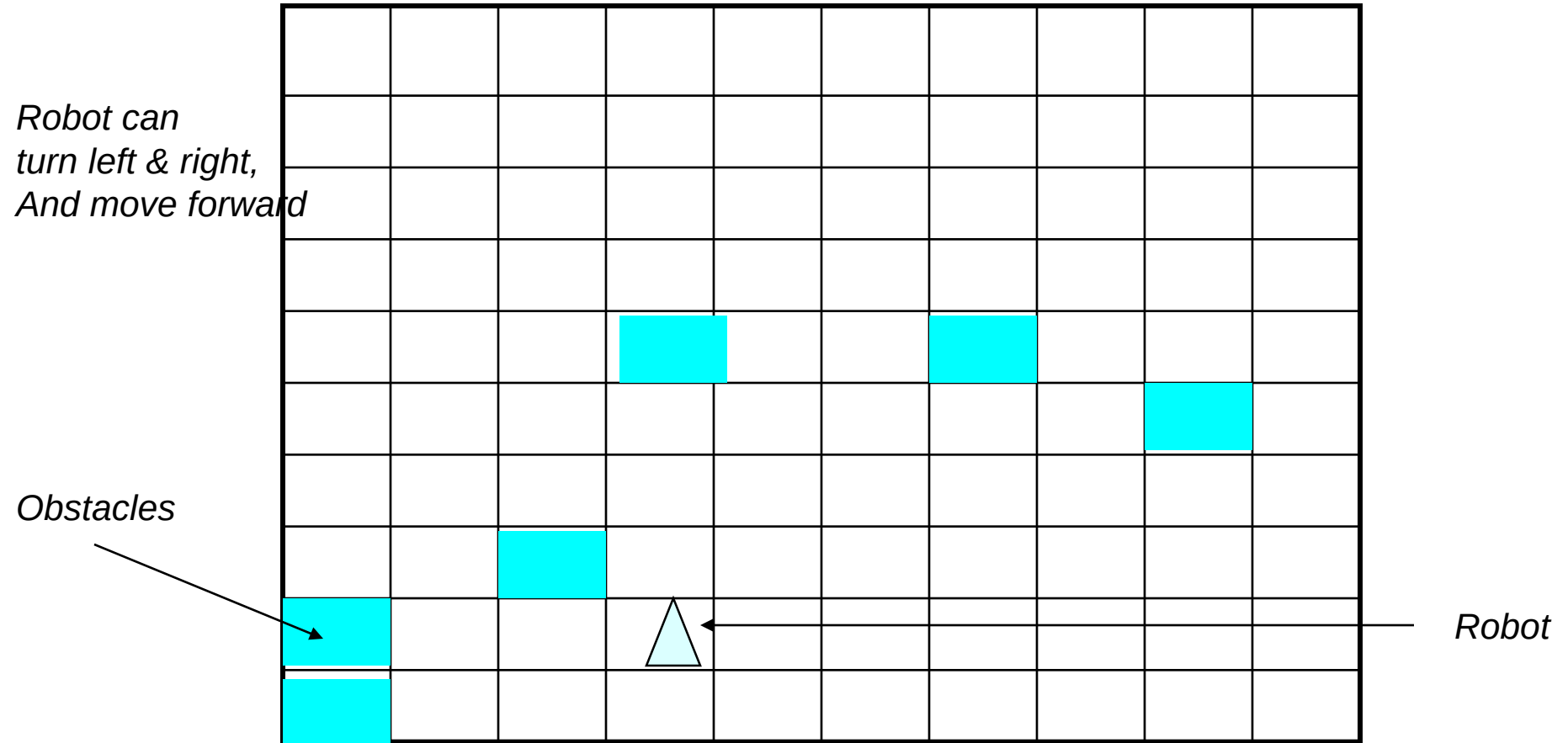
• add the subtree $DTL(Examples_v, TargetAttribute, Attributes - \{A\})$

• **End For**

• **End begin**

• **Return** *Root*

Example



Example

	Left Sensor	Right Sensor	Forward Sensor	Back Sensor	Previous Action	Action
X1	Obstacle	Free	Obstacle	Free	moveForward	TurnRight
X2	Free	Free	Obstacle	Free	TurnLeft	TurnLeft
X3	Free	Obstacle	Free	Free	moveForward	MoveForward
X4	Free	Free	Free	Obstacle	TurnLeft	MoveForward
X5	Free	Free	Free	Free	TurnRight	MoveForward
X6	Free	Free	Free	Obstacle	TurnRight	moveForward

Example

$$\text{Entropy} = -1/6 \cdot \log_2(1/6) - 1/6 \cdot \log_2(1/6) - 4/6 \cdot \log_2(4/6) = 1.25$$

$$\begin{aligned} \text{Gain}(\text{LeftSensor}) &= \text{Entropy} - 2/6 \cdot \text{Entropy}(\text{LeftSensor}=\text{obstacle}) - \\ &4/6 \cdot \text{Entropy}(\text{LeftSensor}=\text{free}) \\ &= 1.25 - 2/6 \cdot 1 - 4/6 \cdot 0.811 = 0.326 \end{aligned}$$

$$\begin{aligned} \text{Gain}(\text{RightSensor}) &= \text{Entropy} - 2/6 \cdot \text{Entropy}(\text{RightSensor}=\text{obstacle}) - \\ &4/6 \cdot \text{Entropy}(\text{RightSensor}=\text{free}) \\ &= 1.25 - 2/6 \cdot 0 - 4/6 \cdot 1.5 = 0.25 \end{aligned}$$

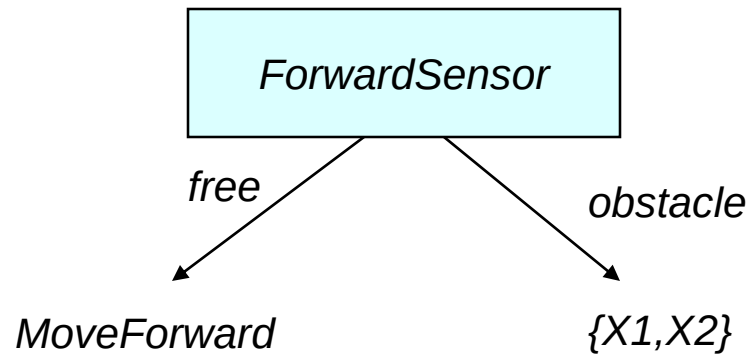
$$\begin{aligned} \text{Gain}(\text{ForwardSensor}) &= \text{Entropy} - 2/6 \cdot \text{Entropy}(\text{ForwardSensor}=\text{obstacle}) - \\ &4/6 \cdot \text{Entropy}(\text{ForwardSensor}=\text{Free}) \\ &= 1.25 - 2/6 \cdot 1 - 4/6 \cdot 0 = .917 \end{aligned}$$

$$\begin{aligned} \text{Gain}(\text{BackSensor}) &= \text{Entropy} - 2/6 \cdot \text{Entropy}(\text{BackSensor}=\text{obstacle}) - \\ &4/6 \cdot \text{Entropy}(\text{BackSensor}=\text{free}) \\ &= 1.25 - 2/6 \cdot 0 - 4/6 \cdot 1.5 = 0.25 \end{aligned}$$

$$\begin{aligned} \text{Gain}(\text{PreviousAction}) &= \text{Entropy} - 2/6 \cdot \text{Entropy}(\text{PreviousAction}=\text{MoveForward}) - \\ &2/6 \cdot \text{Entropy}(\text{PreviousAction}=\text{TurnLeft}) - \\ &2/6 \cdot \text{Entropy}(\text{PreviousAction}=\text{TurnRight}) \\ &= 1.25 - 2/6 \cdot 1 - 2/6 \cdot 1 - 2/6 \cdot 0 \\ &= 0.58 \end{aligned}$$

Select ForwardSensor

Example



Select ForwardSensor

Example

$$\text{Entropy}(\{X1, X2\}) = -1/2 * \log_2(1/2) - 1/2 * \log_2(1/2) = 1$$

$$\text{Gain}(\text{LeftSensor})$$

$$= 1 - 1/2 * \text{Entropy}(\text{LeftSensor} = \text{free}) -$$

$$1/2 * \text{Entropy}(\text{LeftSensor} = \text{obstacle})$$

$$= 1 - 1/2 * 1 - 1/2 * 0 - 1/2 * 0$$

$$= 1$$

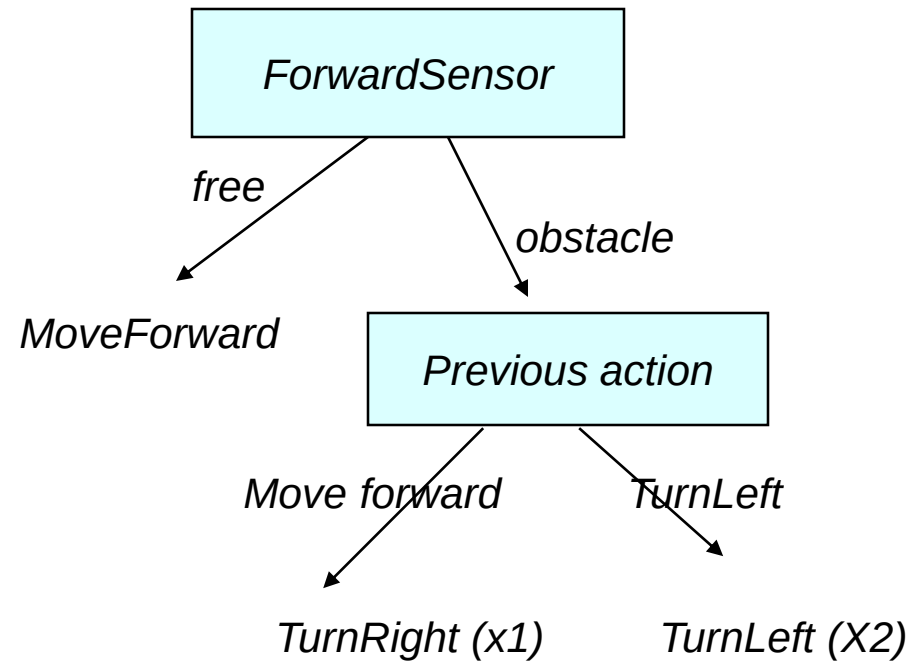
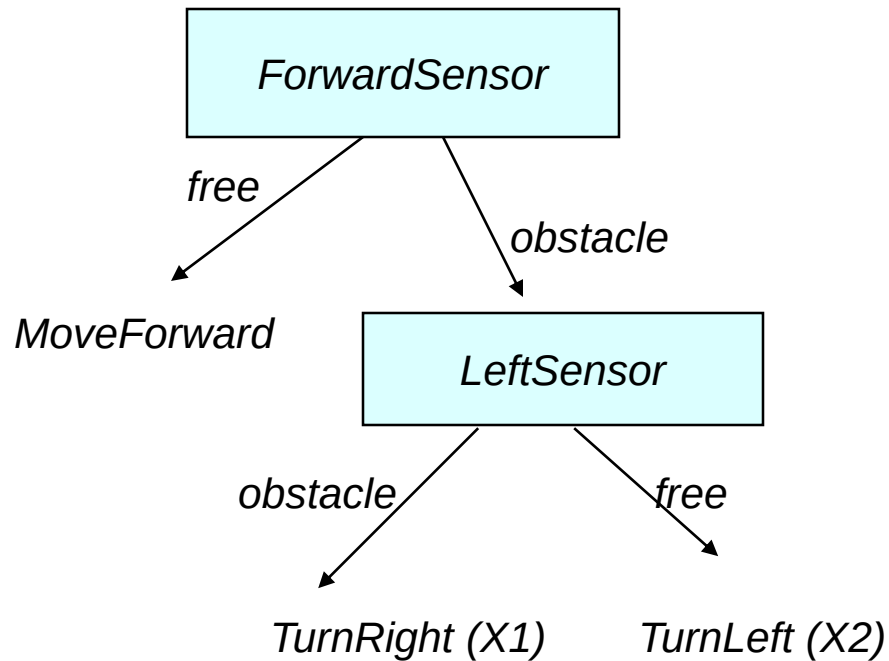
$$\text{Gain}(\text{RightSensor}) = 1 - 1 * \text{Entropy}(\text{RightSensor} = 0) = 1 - 1 = 0$$

$$\text{Gain}(\text{BackSensor}) = 0$$

$$\text{Gain}(\text{PreviousAction}) = 1$$

Either LeftSensor or PreviousAction depending on the execution order

Example



Bias

Babylon 5 universe

	Race	Name	BeenToB5	GoodPerson
D1	Minbari	Delenn	Yes	Yes
D2	Minbari	Draal	Yes	Yes
D3	Human	Morden	Yes	No
D4	Narn	G'Kar	Yes	Yes
D5	Human	Sheridan	Yes	Yes

$$p-=0.2 \quad p+=0.8$$

$$Entropy(S)=-0.2*\log_2 0.2-0.8*\log_2 0.8=0.72$$

Bias

$$V_{\text{race}} = \{\text{Minbari}, \text{Human}, \text{Narn}\}$$

$$S_{\text{minbari}} = \{D1(+), D2(+)\}$$

$$S_{\text{human}} = \{D3(-), D5(+)\}$$

$$S_{\text{narn}} = \{D4(+)\}$$

Conditional Entropy=

$$2/5 * \text{Entropy}(S_{\text{minbari}}) + 2/5 * \text{Entropy}(S_{\text{human}}) + 1/5 * \text{Entropy}(S_{\text{narn}})$$

$$\text{Entropy}(S_{\text{minbari}}) = 0$$

$$\text{Entropy}(S_{\text{human}}) = 1$$

$$\text{Entropy}(S_{\text{narn}}) = 0$$

$$\text{Gain}(S, \text{Race}) = 0.72 - 2/5 = 0.32$$

Bias

$$V_{name} = \{Delenn, Draal, Morden, G'Kar, Sheridan\}$$

$$S_{Delenn} = \{D1(+)\}$$

$$S_{Draal} = \{D2(+)\}$$

$$S_{Morden} = \{D3(-)\}$$

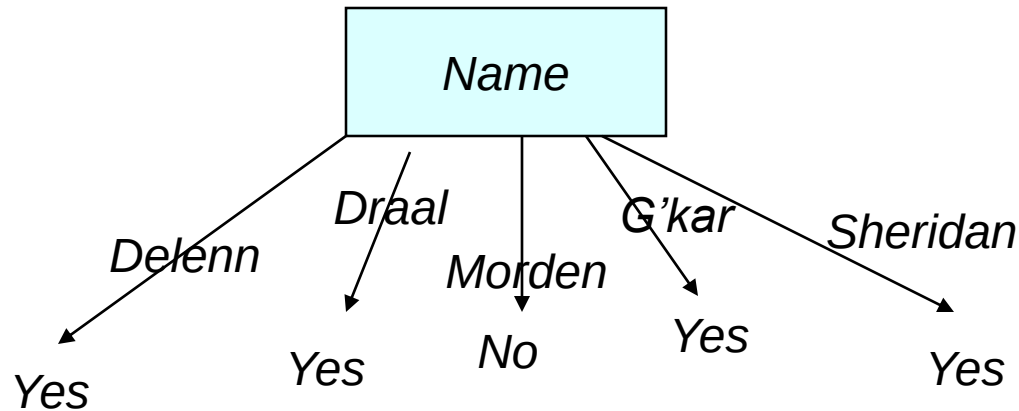
$$S_{G'Kar} = \{D4(+)\}$$

$$S_{Sheridan} = \{D5(+)\}$$

The entropies of all subsets are 0 so Conditional Entropy=0

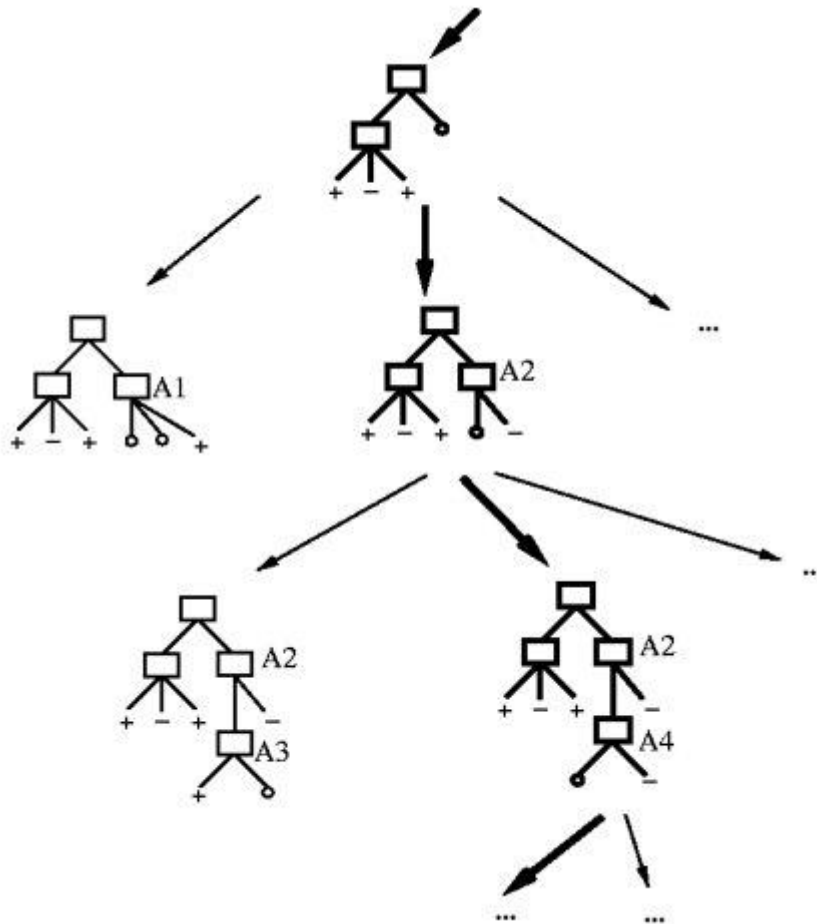
$$Gain(S, Name) = 0.72 - 0 = .72$$

Bias



Inductive bias: select the shortest trees. Trees that place high information gain attributes close to the root are preferred over those that do not.

Hypothesis Space



ID3 is a heuristic to select a tree in top-down fashion.

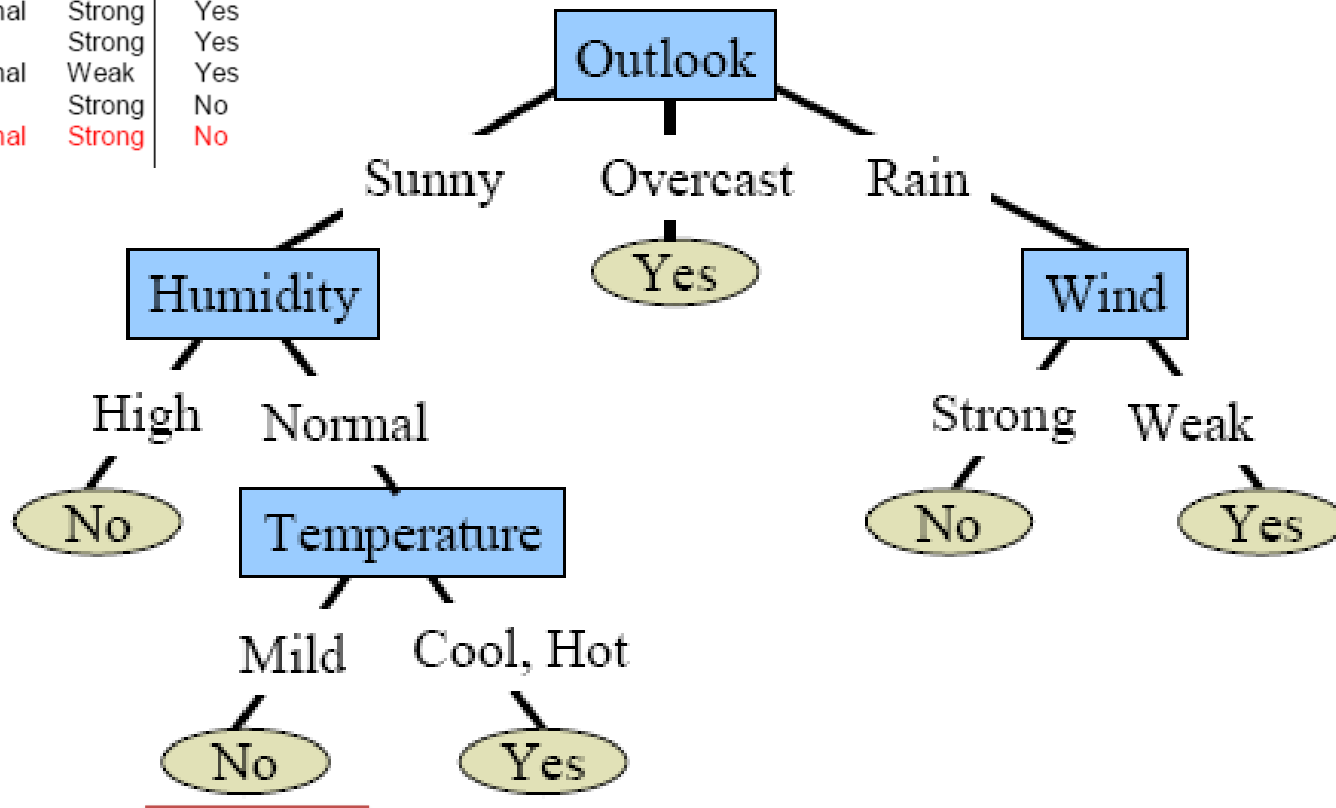
ID3 can be characterized as searching a space of hypotheses for one that fits the training examples

Overfitting and Occam's Razor

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Hot	Normal	Weak	Yes
D10	Rain	Mild	Normal	Strong	Yes
D11	Sunny	Cool	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No
D15	Sunny	Mild	Normal	Strong	No

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Hot	Normal	Weak	Yes
D10	Rain	Mild	Normal	Strong	Yes
D11	Sunny	Cool	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No
D15	Sunny	Mild	Normal	Strong	No

Effect on Our Tree



Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D15	Sunny	Mild	Normal	Strong	No

Occam's razor

- "Non sunt multiplicanda entia praeter necessitatem."
 - William of Occam (1285-1347)
- In English: Entities are not to be multiplied beyond necessity (law of parsimony)
- In science: Prefer simpler hypotheses over more complex ones (Occam's razor)
- "Everything should be made as simple as possible, but not simpler" Albert Einstein (1879-1955)

Decision Tree

C4.5

Information-Gain Ratio

Name has the most information Gain in the training set (generating more sets than others).

Another measure is GainRatio (信息增益比)

Let $H_A(S) = - \sum_{i=1}^v (|S_i|/|S|) \log_2 (|S_i|/|S|)$, where v is the number of values of Attribute A. how broadly and uniformly the attribute splits the data.

$$\text{GainRatio}(S,A) = G(S,A) / H_A(S)$$

Algorithm

输入: 数据集 D 特征集 A

输出: 决策树 T

1. 如果 D 中的所有实例均属于同一类 C_k , 则 T 为单结点树, 并将 C_k 做为此结点的类, 返回 T ;
2. 如果 A 为空集, 则 T 为单结点树, 并将 D 中拥有实例数最大的类 C_k 作为该结点的类, 返回 T ;
3. 否则:
 1. 计算 A 中每个特征的信息增益比, 选择信息增益比最大的特征 A_g (若特征值为连续值, 处理过程见1.2.3.2节);
 2. 遍历特征 A_g 可能取的每一个值 v_i , 令 $\text{Examples}(v_i)$ 为 D 中特征 A_g 值为 v_i 的子集, 如果 $\text{Examples}(v_i)$ 为空, 则建立一个叶子结点, 将 D 中所拥有实例数最大的 C_k 作为此结点的类标记; 如果 $\text{Examples}(v_i)$ 不为空集, 则对数据集 $\text{Examples}(v_i)$ 和以 $A - \{A_g\}$ 为特征集继续递归的执行C4.5构建决策树算法;
4. 返回结点 T 。

Example

Race	Name	BeenToB5	GoodPerson
Minbari	Delenn	Yes	Yes
Minbari	Draal	Yes	Yes
Human	Morden	Yes	No
Human	Bester	Yes	No
Human	Sinclair	Yes	Yes
Minbari	Lennier	Yes	Yes
Minbari	Callenn	Yes	Yes
Human	Sheridan	Yes	Yes

$$p-=.25 \quad p+=.75$$

$$Entropy(S)=-0.25*\log_2 0.25-0.75*\log_2 0.75=0.811$$

Race	Name	BeenToB5	GoodPerson
Minbari	Delenn	Yes	Yes
Minbari	Draal	Yes	Yes
Human	Morden	Yes	No
Human	Bester	Yes	No
Human	Sinclair	Yes	Yes
Minbari	Lennier	Yes	Yes
Minbari	Callenn	Yes	Yes
Human	Sheridan	Yes	Yes

$Gain(S, Race)$

$$= 0.811 - [4/8 * (-4/4 \log_2(4/4)) + 4/8 * (-2/4 \log_2(2/4) - 2/4 \log_2(2/4))] = 0.811 - 1/2$$

$$= 0.811 - 1/2$$

$$= 0.311$$

$$H_{Race}(S) = -4/8 * \log_2(4/8) - 4/8 * \log_2(4/8) = 1$$

$$GainRatio(S, Race) = Gain(S, Race) / H_{Race}(S) = \mathbf{0.311}$$

Race	Name	BeenToB5	GoodPerson
Minbari	Delenn	Yes	Yes
Minbari	Draal	Yes	Yes
Human	Morden	Yes	No
Human	Bester	Yes	No
Human	Sinclair	Yes	Yes
Minbari	Lennier	Yes	Yes
Minbari	Callenn	Yes	Yes
Human	Sheridan	Yes	Yes

$$Gain(S, Name) = 0.811 - 0 = 0.811$$

$$H_{Name}(S)$$

$$= -1/8 * \log_2(1/8) - 1/8 * \log_2(1/8) - 1/8 * \log_2(1/8) - 1/8 * \log_2(1/8) - 1/8 * \log_2(1/8) - 1/8 * \log_2(1/8) - 1/8 * \log_2(1/8) - 1/8 * \log_2(1/8)$$

$$= 3$$

$GainRatio(S, Name) = 0.27 \longrightarrow$ So select **Race** as first branch

Continuous-Valued Attributes

Dynamically define new discrete-valued attributes that partition the continuous attribute into a discrete set of **intervals**

Example: Suppose we have training examples with the following Temp values:

Temp	40	48	60	72	80	90
<i>PlayTennis</i>	No	No	Yes	Yes	Yes	No

Split into two intervals: $Temp \leq val$ and $Temp > val$

This defines a new boolean attribute $Temp_{>val}$

How to choose *val threshold*?

Consider boundary cases $val = (48+60)/2 = 54$ and $val = (80+90)/2 = 85$

Possible attributes: $Temp_{>54}$, $Temp_{>85}$

Choose attribute with largest *information gain* (as before): $Temp_{>54}$

This approach can be generalized to multiple thresholds.

表 4.3 西瓜数据集 3.0

编号	色泽	根蒂	敲声	纹理	脐部	触感	密度	含糖率	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	0.697	0.460	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	0.774	0.376	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	0.634	0.264	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	0.608	0.318	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	0.556	0.215	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	0.403	0.237	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	0.481	0.149	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	0.437	0.211	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	0.666	0.091	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	0.243	0.267	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	0.245	0.057	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	0.343	0.099	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	0.639	0.161	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	0.657	0.198	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	0.360	0.370	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	0.593	0.042	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	0.719	0.103	否

对属性“密度”，在决策树学习开始时，根结点包含的 17 个训练样本在该属性上取值均不同。根据式(4.7)，该属性的候选划分点集合包含 16 个候选值： $T_{\text{密度}} = \{0.244, 0.294, 0.351, 0.381, 0.420, 0.459, 0.518, 0.574, 0.600, 0.621, 0.636, 0.648, 0.661, 0.681, 0.708, 0.746\}$ 。由式(4.8)可计算出属性“密度”的信息增益为 0.262，对应于划分点 0.381。

对属性“含糖率”，其候选划分点集合也包含 16 个候选值： $T_{\text{含糖率}} = \{0.049, 0.074, 0.095, 0.101, 0.126, 0.155, 0.179, 0.204, 0.213, 0.226, 0.250, 0.265, 0.292, 0.344, 0.373, 0.418\}$ 。类似的，根据式(4.8)可计算出其信息增益为 0.349，对应于划分点 0.126。

再由 4.2.1 节可知，表 4.3 的数据上各属性的信息增益为

$$\text{Gain}(D, \text{色泽}) = 0.109; \quad \text{Gain}(D, \text{根蒂}) = 0.143;$$

$$\text{Gain}(D, \text{敲声}) = 0.141; \quad \text{Gain}(D, \text{纹理}) = 0.381;$$

$$\text{Gain}(D, \text{脐部}) = 0.289; \quad \text{Gain}(D, \text{触感}) = 0.006;$$

$$\text{Gain}(D, \text{密度}) = 0.262; \quad \text{Gain}(D, \text{含糖率}) = 0.349.$$

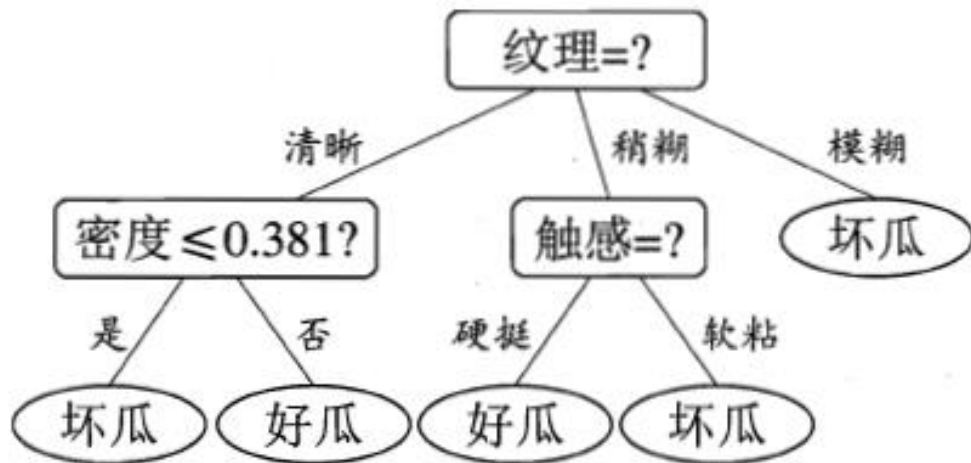


图 4.8 在西瓜数据集 3.0 上基于信息增益生成的决策树

Unknown Attribute Values

- Assign most common value in tree
- Assign most common value among training set
- Assign probability to each possible value of A

表 4.4 西瓜数据集 2.0 α

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	—	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	—	是
3	乌黑	蜷缩	—	清晰	凹陷	硬滑	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	—	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	—	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
8	乌黑	稍蜷	浊响	—	稍凹	硬滑	是
9	乌黑	—	沉闷	稍糊	稍凹	硬滑	否
10	青绿	硬挺	清脆	—	平坦	软粘	否
11	浅白	硬挺	清脆	模糊	平坦	—	否
12	浅白	蜷缩	—	模糊	平坦	软粘	否
13	—	稍蜷	浊响	稍糊	凹陷	硬滑	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	—	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	—	沉闷	稍糊	稍凹	硬滑	否

我们需解决两个问题: (1) 如何在属性值缺失的情况下进行划分属性选择?
(2) 给定划分属性, 若样本在该属性上的值缺失, 如何对样本进行划分?

给定训练集 D 和属性 a , 令 $\tilde{D}$ 表示 D 中在属性 a 上没有缺失值的样本子集. 对问题(1), 显然我们仅可根据 $\tilde{D}$ 来判断属性 a 的优劣. 假定属性 a 有 V 个可取值 $\{a^1, a^2, \dots, a^V\}$, 令 $\tilde{D}^v$ 表示 $\tilde{D}$ 中在属性 a 上取值为 a^v 的样本子集, $\tilde{D}_k$ 表示 $\tilde{D}$ 中属于第 k 类 ($k = 1, 2, \dots, |\mathcal{Y}|$) 的样本子集, 则显然有 $\tilde{D} = \bigcup_{k=1}^{|\mathcal{Y}|} \tilde{D}_k$, $\tilde{D} = \bigcup_{v=1}^V \tilde{D}^v$. 假定我们为每个样本 $\mathbf{x}$ 赋予一个权重 $w_{\mathbf{x}}$, 并定义

$$\rho = \frac{\sum_{\mathbf{x} \in \tilde{D}} w_{\mathbf{x}}}{\sum_{\mathbf{x} \in D} w_{\mathbf{x}}}, \quad (4.9)$$

$$\tilde{p}_k = \frac{\sum_{\mathbf{x} \in \tilde{D}_k} w_{\mathbf{x}}}{\sum_{\mathbf{x} \in \tilde{D}} w_{\mathbf{x}}} \quad (1 \leq k \leq |\mathcal{Y}|), \quad (4.10)$$

$$\tilde{r}_v = \frac{\sum_{\mathbf{x} \in \tilde{D}^v} w_{\mathbf{x}}}{\sum_{\mathbf{x} \in \tilde{D}} w_{\mathbf{x}}} \quad (1 \leq v \leq V). \quad (4.11)$$

直观地看, 对属性 a , ρ 表示无缺失值样本所占的比例, $\tilde{p}_k$ 表示无缺失值样本中第 k 类所占的比例, $\tilde{r}_v$ 则表示无缺失值样本中在属性 a 上取值 a^v 的样本所占的比例. 显然, $\sum_{k=1}^{|\mathcal{Y}|} \tilde{p}_k = 1$, $\sum_{v=1}^V \tilde{r}_v = 1$.

基于上述定义, 我们可将信息增益的计算式(4.2)推广为

$$\begin{aligned} \text{Gain}(D, a) &= \rho \times \text{Gain}(\tilde{D}, a) \\ &= \rho \times \left(\text{Ent}(\tilde{D}) - \sum_{v=1}^V \tilde{r}_v \text{Ent}(\tilde{D}^v) \right), \end{aligned} \quad (4.12)$$

其中由式(4.1), 有

$$\text{Ent}(\tilde{D}) = - \sum_{k=1}^{|\mathcal{Y}|} \tilde{p}_k \log_2 \tilde{p}_k.$$

对问题(2), 若样本 $\mathbf{x}$ 在划分属性 a 上的取值已知, 则将 $\mathbf{x}$ 划入与其取值对应的子结点, 且样本权值在子结点中保持为 $w_{\mathbf{x}}$. 若样本 $\mathbf{x}$ 在划分属性 a 上的取值未知, 则将 $\mathbf{x}$ 同时划入所有子结点, 且样本权值在与属性值 a^v 对应的子结点中调整为 $\tilde{r}_v \cdot w_{\mathbf{x}}$; 直观地看, 这就是让同一个样本以不同的概率划入到不同的子结点中去.

表 4.4 西瓜数据集 2.0 α

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	—	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	—	是
3	乌黑	蜷缩	—	清晰	凹陷	硬滑	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	—	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	—	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
8	乌黑	稍蜷	浊响	—	稍凹	硬滑	是
9	乌黑	—	沉闷	稍糊	稍凹	硬滑	否
10	青绿	硬挺	清脆	—	平坦	软粘	否
11	浅白	硬挺	清脆	模糊	平坦	—	否
12	浅白	蜷缩	—	模糊	平坦	软粘	否
13	—	稍蜷	浊响	稍糊	凹陷	硬滑	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	—	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	—	沉闷	稍糊	稍凹	硬滑	否

在学习开始时, 根结点包含样本集 D 中全部 17 个样例, 各样例的权值均为 1. 以属性“色泽”为例, 该属性上无缺失值的样例子集 $\tilde{D}$ 包含编号为 $\{2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 14, 15, 16, 17\}$ 的 14 个样例. 显然, $\tilde{D}$ 的信息熵为

$$\begin{aligned}\text{Ent}(\tilde{D}) &= - \sum_{k=1}^2 \tilde{p}_k \log_2 \tilde{p}_k \\ &= - \left(\frac{6}{14} \log_2 \frac{6}{14} + \frac{8}{14} \log_2 \frac{8}{14} \right) = 0.985 .\end{aligned}$$

令 $\tilde{D}^1$, $\tilde{D}^2$ 与 $\tilde{D}^3$ 分别表示在属性“色泽”上取值为“青绿”“乌黑”以及“浅白”的样本子集, 有

$$\begin{aligned}\text{Ent}(\tilde{D}^1) &= - \left(\frac{2}{4} \log_2 \frac{2}{4} + \frac{2}{4} \log_2 \frac{2}{4} \right) = 1.000 , \\ \text{Ent}(\tilde{D}^2) &= - \left(\frac{4}{6} \log_2 \frac{4}{6} + \frac{2}{6} \log_2 \frac{2}{6} \right) = 0.918 ,\end{aligned}$$

$$\text{Ent}(\tilde{D}^3) = - \left(\frac{0}{4} \log_2 \frac{0}{4} + \frac{4}{4} \log_2 \frac{4}{4} \right) = 0.000 ,$$

因此, 样本子集 $\tilde{D}$ 上属性“色泽”的信息增益为

$$\begin{aligned} \text{Gain}(\tilde{D}, \text{色泽}) &= \text{Ent}(\tilde{D}) - \sum_{v=1}^3 \tilde{r}_v \text{Ent}(\tilde{D}^v) \\ &= 0.985 - \left(\frac{4}{14} \times 1.000 + \frac{6}{14} \times 0.918 + \frac{4}{14} \times 0.000 \right) \\ &= 0.306 . \end{aligned}$$

于是, 样本集 D 上属性“色泽”的信息增益为

$$\text{Gain}(D, \text{色泽}) = \rho \times \text{Gain}(\tilde{D}, \text{色泽}) = \frac{14}{17} \times 0.306 = 0.252 .$$

类似地可计算出所有属性在 D 上的信息增益:

$$\text{Gain}(D, \text{色泽}) = 0.252; \quad \text{Gain}(D, \text{根蒂}) = 0.171;$$

$$\text{Gain}(D, \text{敲声}) = 0.145; \quad \text{Gain}(D, \text{纹理}) = 0.424;$$

$$\text{Gain}(D, \text{脐部}) = 0.289; \quad \text{Gain}(D, \text{触感}) = 0.006.$$

“纹理”在所有属性中取得了最大的信息增益,被用于对根结点进行划分.划分结果是使编号为 {1,2,3,4,5,6,15} 的样本进入“纹理=清晰”分支,编号为 {7,9,13,14,17} 的样本进入“纹理=稍糊”分支,而编号为 {11,12,16} 的样本进入“纹理=模糊”分支,且样本在各子结点中的权重保持为 1. 需要注意的是,编号为 {8} 的样本在属性“纹理”上出现了缺失值,因此它将同时进入三个分支中,但权重在三个子结点中分别调整为 $\frac{7}{15}$ 、 $\frac{5}{15}$ 和 $\frac{3}{15}$. 编号为 {10} 的样本有类似划分结果.

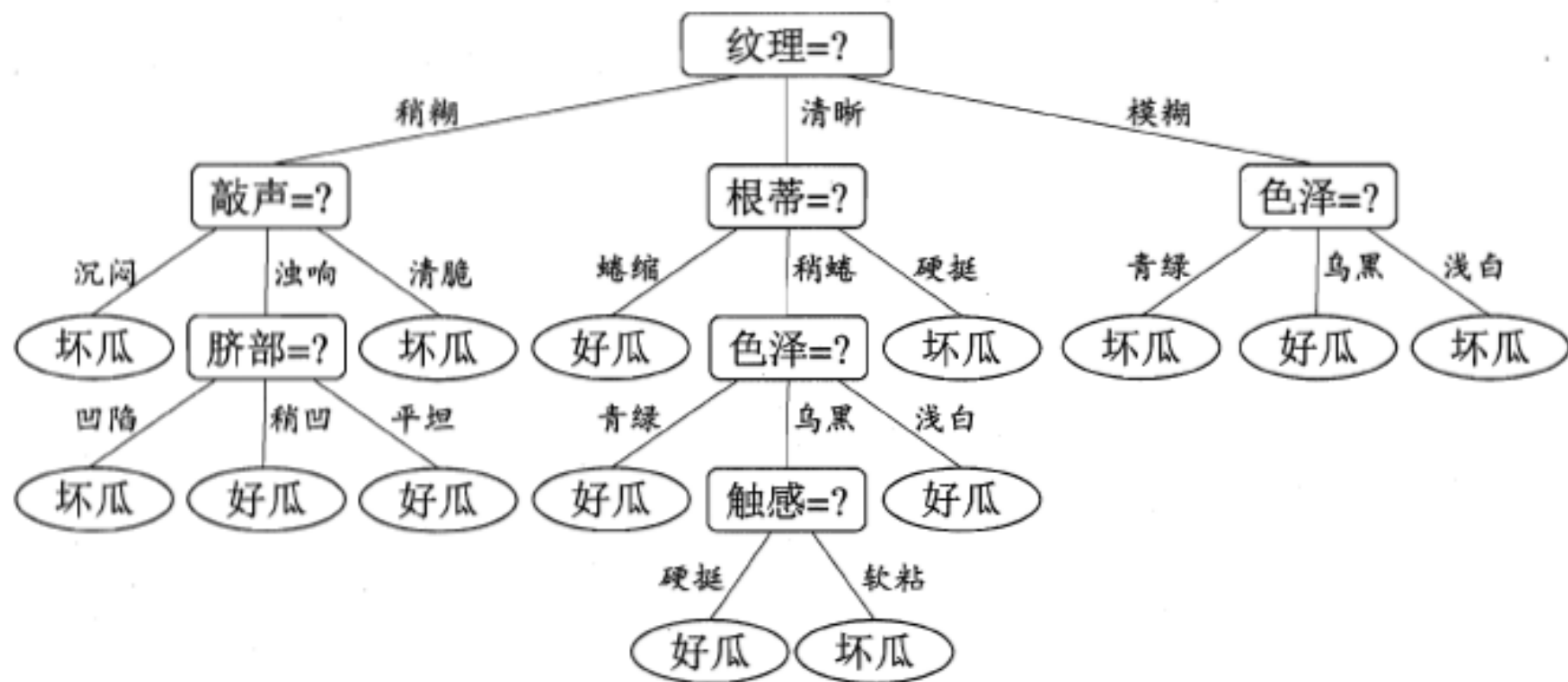


图 4.9 在西瓜数据集 2.0 α 上基于信息增益生成的决策树

Decision Tree

CLASSIFICATION AND REGRESSION TREE (CART) - CLASSIFICATION TREE

Gini Index

- 在K分类问题中, 样本点属于第 k 类的概率为 p_k , 概率分布的Gini index为

$$\text{Gini}(p) = \sum_{k=1}^K p_k(1 - p_k) = 1 - \sum_{k=1}^K p_k^2$$

- 若为二分类问题, 令一个样本点属于第1类的概率为 p , 则此概率分布的Gini index为

$$\text{Gini}(p) = 2p(1 - p)$$

- 给定的数据集 D , 其Gini index定义为

$$\text{Gini}(D) = 1 - \sum_{k=1}^K \left(\frac{|C_k|}{|D|} \right)^2$$

K 是类别的个数, $|D|$ 为数据集 D 中样本的个数, C_k 为数据集 D 中属于类别 k 的样本的集合, $|C_k|$ 为 C_k 中样本的个数。

Gini Index

- 数据集 D 可以根据特征 A 的一个值划分为 D_1 和 D_2 两部分, 则在选取特征 A 的条件下, 集合 D 的基尼指数定义为:

$$\text{Gini}(D, A) = \frac{|D_1|}{|D|} \text{Gini}(D_1) + \frac{|D_2|}{|D|} \text{Gini}(D_2)$$

$\text{Gini}(D)$ 表示了整个数据集 D 的不确定性。

$\text{Gini}(D, A)$ 表示经过属性 A 的一个值分割后的 D 的不确定性。

基尼指数越大, 数据集的不确定性就越大。

A_1	+	-
T	3	1
F	1	4

A_2	+	-
T	2	3
F	2	2

$$\text{Gini}(D, A_1) = \frac{4}{9} \text{Gini}(D_1) + \frac{5}{9} \text{Gini}(D_2)$$

$$\text{Gini}(D_1) = 1 - (3/4)^2 - (1/4)^2$$

$$\text{Gini}(D_2) = 1 - (1/5)^2 - (4/5)^2$$

$$\text{Gini}(D, A_1) = 0.3444$$

$$\begin{aligned} \text{Gini}(D, A_2) &= 5/9[1 - (2/5)^2 - (3/5)^2] \\ &+ 4/9[1 - (2/4)^2 - (2/4)^2] = 0.4889 \end{aligned}$$

Classification Tree

输入: 训练数据集 D

输出: 分类树

从根结点开始, 根据训练数据集递归的对每个结点进行如下操作:

1. 在当前结点对应的数据集上, 计算每一个特征以及其取值的基尼指数。
2. 比较步骤1中的所有特征以及其所有可能的取值对应的基尼指数, 选择**基尼指数最小**的为最优特征和最优切分点;
3. 根据步骤2中选择的最优特征和切分点, 将当前的数据集划分到两个子结点中去;
4. 对步骤3中的两个子结点递归的调用步骤1、2、3, 直至满足停止条件;
5. 返回分类树。

Example

- 给定表中所示的训练数据集，使用CART算法生成分类树。

ID	年龄	有工作	有自己的房子	信贷情况	类别
1	青年	否	否	一般	否
2	青年	否	否	好	否
3	青年	是	否	好	是
4	青年	是	是	一般	是
5	青年	否	否	一般	否
6	中年	否	否	一般	否
7	中年	否	否	好	否
8	中年	是	是	好	是
9	中年	否	是	非常好	是
10	中年	否	是	非常好	是
11	老年	否	是	非常好	是
12	老年	否	是	好	是
13	老年	是	否	好	是
14	老年	是	否	非常好	是
15	老年	否	否	一般	否

- $$\text{Gini}(D, A_1 = \text{青年}) = \frac{5}{15} \left(2 \times \frac{2}{5} \times \left(1 - \frac{2}{5} \right) \right) + \frac{10}{15} \left(2 \times \frac{7}{10} \times \left(1 - \frac{7}{10} \right) \right) = 0.44$$
- $\text{Gini}(D, A_1 = \text{中年}) = 0.48$
- $\text{Gini}(D, A_1 = \text{老年}) = 0.44$
- $\text{Gini}(D, A_2 = \text{是}) = \text{Gini}(D, A_2 = \text{否}) = 0.32$
- $\text{Gini}(D, A_3 = \text{是}) = \text{Gini}(D, A_3 = \text{否}) = 0.27$
- $\text{Gini}(D, A_4 = \text{非常好}) = 0.36$
- $\text{Gini}(D, A_4 = \text{好}) = 0.47$
- $\text{Gini}(D, A_4 = \text{一般}) = 0.32$
- 综合特征 A_1, A_2, A_3, A_4 4个特征各自取值的基尼指数， $\text{Gini}(D, A_3 = \text{是}) = \text{Gini}(D, A_3 = \text{否}) = 0.27$ 最小，所以选择特征 A_3 为最优特征，以及 $A_3 = \text{“是”}$ 或 $A_3 = \text{“否”}$ 为最优切分点。
-

Misclassification Error

- 一个结点的误分类误差 (misclassification error) 定义为:

$$E(D) = 1 - \max_k \left(\frac{|C_k|}{|D|} \right)$$

D 为数据集, $|D|$ 为数据集中样本的个数, C_k 为 D 中属于第 k 类的样本的集合, $|C_k|$ 为 C_k 中样本的个数。

- 二分类问题, 令一个样本点属于第1类的概率为 p , 则此概率分布的误分类误差为:

$$E(D) = 1 - \max(p, 1 - p)$$

Example

- 给出如表中所示数据集 D ，根据误分类误差来选择最优特征。

A	B	C	Number of Instances	
			+	-
T	T	T	5	0
F	T	T	0	20
T	F	T	20	0
F	F	T	0	5
T	T	F	0	0
F	T	F	25	0
T	F	F	0	0
F	F	F	0	25

- 在选择属性对数据集 D 划分以前的误分类误差为：
 $E(D)=1-\max(50/100, 50/100)=50/100$

- 选择属性 A

$$E(D, A = T) = 1 - \max\left(\frac{25}{25}, \frac{0}{25}\right) = 0$$

$$E(D, A = F) = 1 - \max\left(\frac{25}{75}, \frac{50}{75}\right) = \frac{25}{75}$$

$$\Delta_A = E(D) - \frac{25}{100}E(D, A = T) - \frac{75}{100}E(D, A = F) = \frac{25}{100}$$

- 选择属性 B

$$E(D, B = T) = \frac{20}{50}$$

$$E(D, B = F) = \frac{20}{50}$$

$$\Delta_B = E(D) - \frac{50}{100} E(D, B = T) - \frac{50}{100} E(D, B = F) = \frac{10}{100}$$

- 选择属性 C

$$E(D, C = T) = \frac{25}{50}$$

$$E(D, C = F) = \frac{25}{50}$$

$$\Delta_C = E(D) - \frac{50}{100} E(D, C = T) - \frac{50}{100} E(D, C = F) = \frac{0}{100}$$

- 上述步骤可得 Δ_A 最大, 因此选择属性 A 为最优特征。

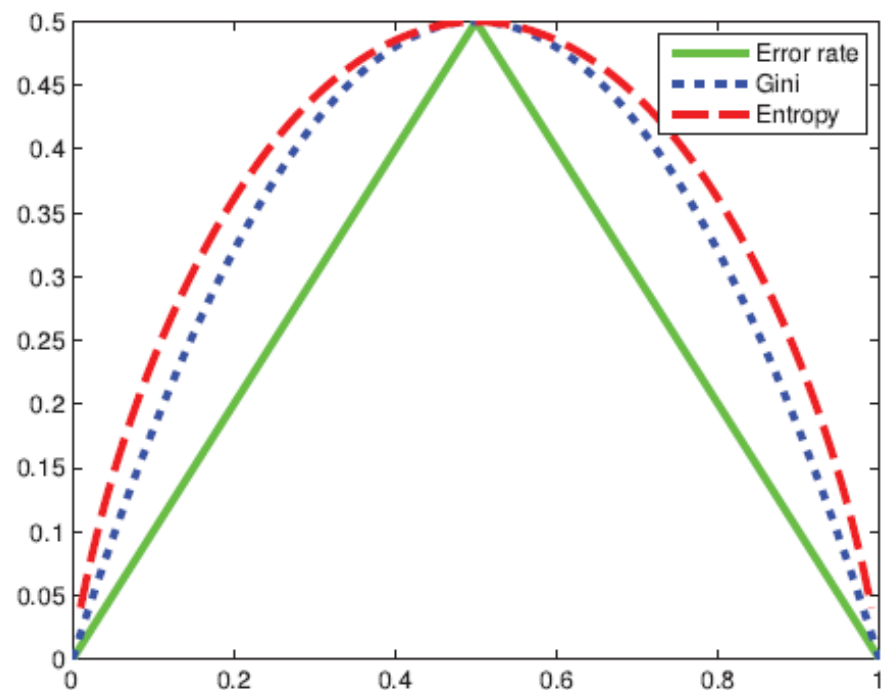


Figure 16.3 Node impurity measures for binary classification. The horizontal axis corresponds to p , the probability of class 1. The entropy measure has been rescaled to pass through $(0.5, 0.5)$. Based on Figure 9.3 of (Hastie et al. 2009). Figure generated by giniDemo.

Decision Tree

CLASSIFICATION AND REGRESSION TREE (CART) - REGRESSION TREE

Least Squares Regression Tree

- X 与 Y 分别为输入和输出变量, Y 是连续变量, 给定训练数据集

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

- 假设将特征空间划分为 M 个区域, 分别为 $R_1, R_2, R_3, \dots, R_M$, 每一个区域对应回归树中的一个叶子结点, 并且在每个区域对应一个固定的输出值 c_m , 回归树模型可以表示为:

$$f(x) = \sum_{m=1}^M c_m I(x \in R_m)$$

I 为指示函数, 即当实例 x 在 R_m 区域中, 则为1, 否则为0。

- 为找到一个合适的切分位置，使用平方误差来计算某个区域对于训练数据的预测误差。
- 若计算一个区域 t 对应的平方误差，计算公式如下：

$$R(d) = \frac{1}{N} \sum_n (y_n - f(x_n))^2$$

当 $f(x_n)$ 取区域 t 中的实例 y 值的平均数时，上式会取最小值，即：

$$f(x_n) = \bar{y}(t) = \frac{1}{N(t)} \sum_{x_n \in t} y_n$$

- 将一个叶子结点对应的输出值设置为此区域中所有实例输出值的平均值：

$$\hat{c}_m = \frac{\sum_i y_i I(x \in R_m)}{\sum_i I(x \in R_m)}$$

R_m 为特征空间中的一个区域， I 为指示函数， y_i 表示第 i 个实例的取值。

- 若要确定一个叶子结点，停止对特征空间的划分，可以通过设定简单的规则来决定是否停止划分。

- 根据切分变量和切分点, 可以将数据集划分为两个区域:

$$R_1(j, s) = \{x | x^{(j)} \leq s\} \quad \text{和} \quad R_2(j, s) = \{x | x^{(j)} > s\}$$

- 遍历所有的切分变量 j , 及所有可能的取值 s 来作为切分变量和切分点, 找到可以使划分后的区域具有最小的预测误差的情况

$$\min_{j, s} \left[\min_{c_1} \sum_{x_i \in R_1(j, s)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j, s)} (y_i - c_2)^2 \right]$$

- 令

$$\hat{c}_1 = \text{ave}(y_i | x_i \in R_{1(j,s)}) \quad \hat{c}_2 = \text{ave}(y_i | x_i \in R_{2(j,s)})$$

- 每个区域的预测误差取最小值, 因此可以将上述公式简化为:

$$\sum_{i: x_j \leq s} (y_i - \hat{c}_1)^2 + \sum_{i: x_i > s} (y_i - \hat{c}_2)^2$$

- 当遍历完所有的切分变量和切分点, 便可以找到此时最优的 j 与 s , 依此将特征空间划分为两个区域, 再对每个区域上重复上述划分过程, 直到满足停止条件。

Algorithm

输入: 训练数据集 D

输出: 回归树 $f(x)$

1. 遍历切分变量 j , 以及对固定的切分变量 j 扫描其对应的切分点 s , 选择可以使如下公式取最小值的 j 与 s 作为最优的切分变量 j 和切分点 s ;

$$\min_{j,s} \left[\min_{c_1} \sum_{x_i \in R_1(j,s)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j,s)} (y_i - c_2)^2 \right] \text{根}$$

2. 据步骤(1)中选定的最优切分变量 j 和切分点 s , 将特征空间划分为两个区域:

$$R_1(j, s) = \{x | x^{(j)} \leq s\} , \quad R_2(j, s) = \{x | x^{(j)} > s\}$$

每个区域对应的输出值为：

$$\hat{c}_m = \frac{1}{N_m} \sum_{x_i \in R_m(j, s)} y_i , \quad x \in R_m, \quad m = 1, 2$$

m 为第 m 个区域；

Example

- 用平方误差损失准则生成一个二叉回归树

x_i	1	2	3	4	5	6	7	8	9	10
y_i	4.50	4.75	4.91	5.34	5.80	7.05	7.90	8.23	8.70	9.00

Pros and cons of trees

- Pros
 - easy to interpret
 - easily handle mixed discrete and continuous inputs
 - insensitive to monotone transformations of the inputs
 - perform automatic variable selection
 - relatively robust to outliers
 - scale well to large data sets
 - can be modified to handle missing inputs

- Cons

- Do not predict very accurately compared to other kinds of model.
- are **unstable**: small changes to the input data can have large effects on the structure of the tree.
- are high variance estimators.

Decision Tree

PRUNING

How can we avoid overfitting

- 1. Prepruning:** Stop growing tree during the training when it is determined that there is not enough data to make reliable choices.
- 2. Postpruning:** Grow whole tree but then remove the branches that do not contribute good overall performance.

Prepruning

1. 设定最大深度。
2. 当决策树的生长已经达到最大的深度时，则停止其生长，直接设置为叶子结点；
3. 设定一个信息增益或其他数据集纯度计算方法的阈值。
4. 当从某个结点继续生长之后，若得到的增益小于设定的阈值时，停止树的生长，直接设置为叶子结点。但是在实际情况中，阈值的取值往往不容易设定；

5. 设定结点最小实例的个数。
6. 若达到一个结点的实例的个数小于设定的最小实例的个数，则停止树的成长，直接将其设为叶子结点；
7. 如果达到某一个结点的实例的特征向量相同，但是可能在数据集中他们所属的类别不同，此时也可以停止决策树的生长，以解决数据冲突的问题；

表 4.2 西瓜数据集 2.0 划分出的训练集(双线上部)与验证集(双线下部)

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否
编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否



图 4.5 基于表 4.2 生成的未剪枝决策树

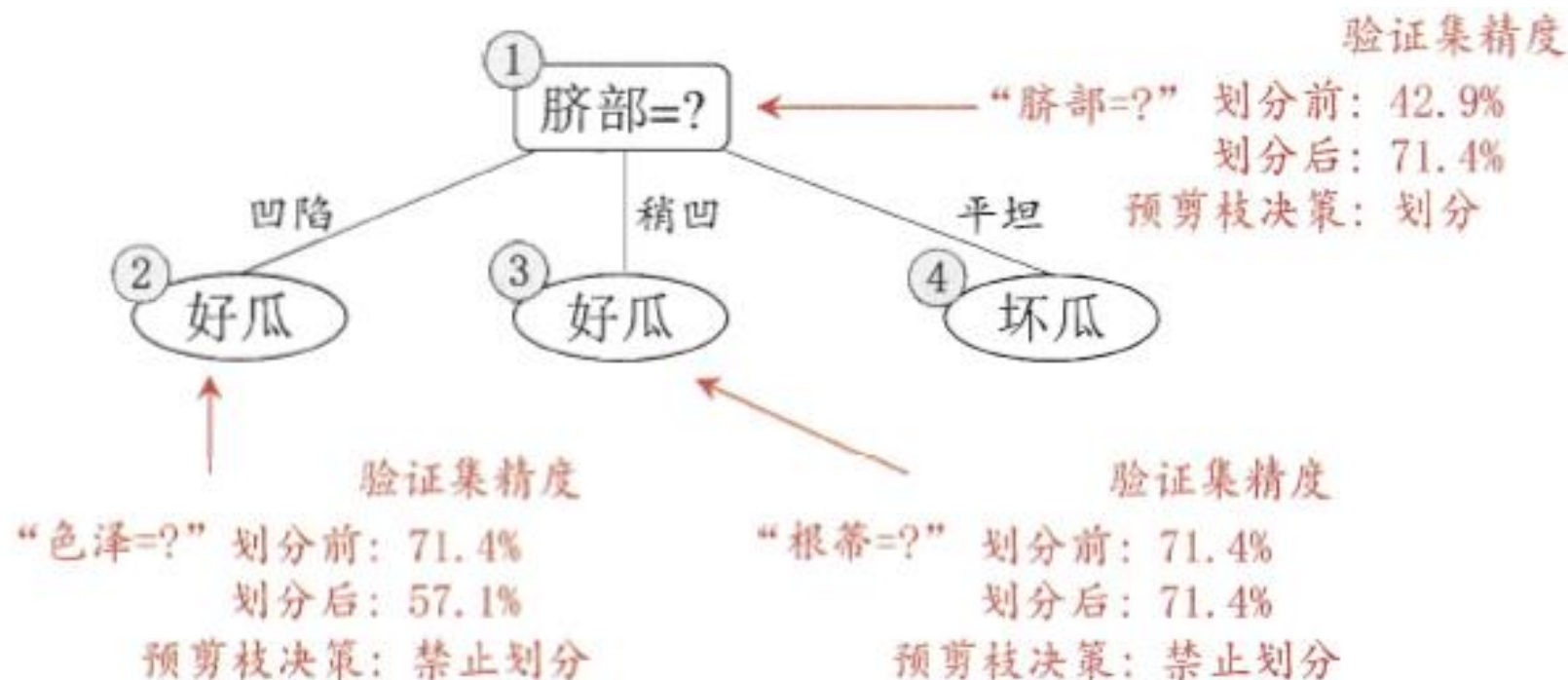


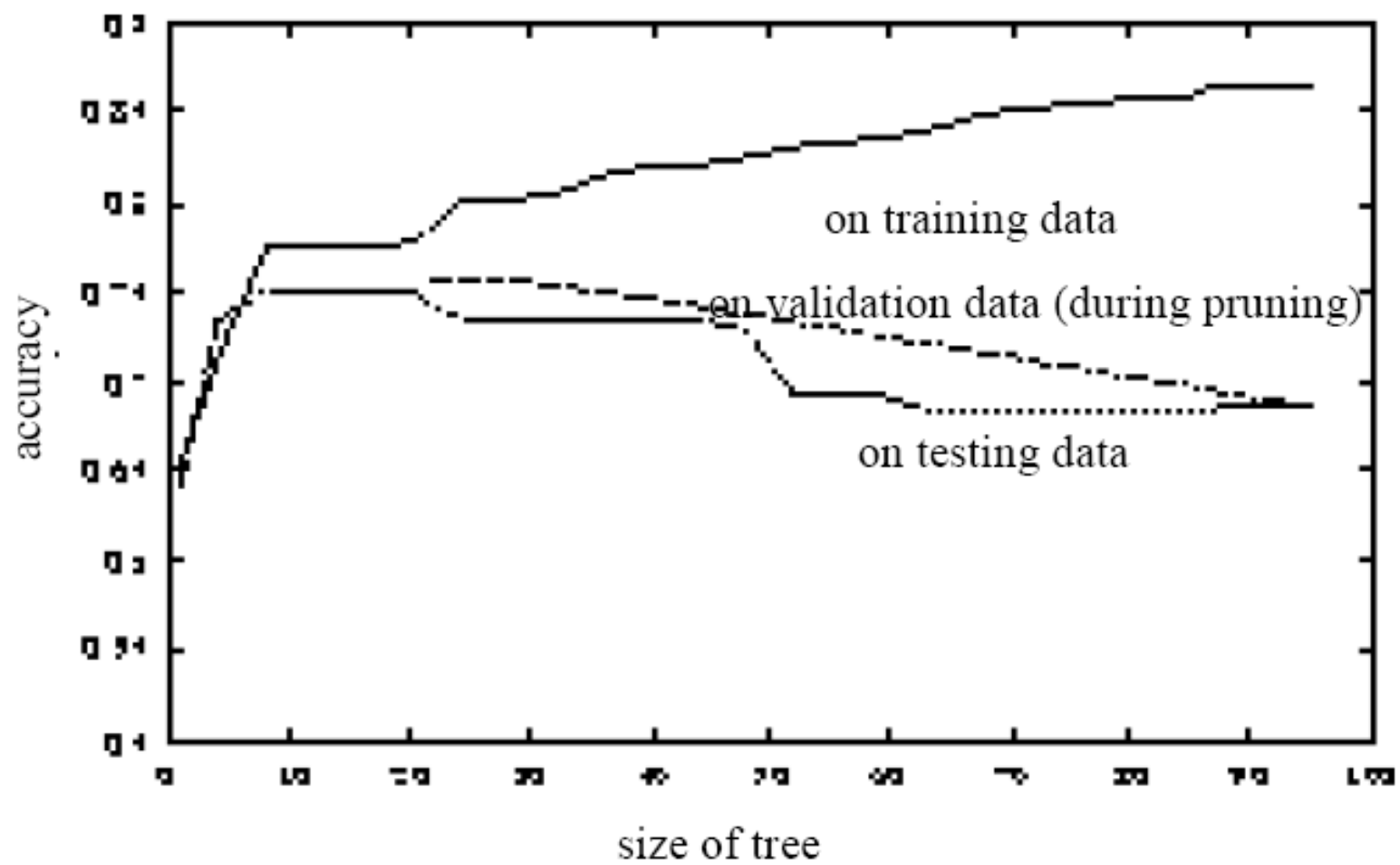
图 4.6 基于表 4.2 生成的预剪枝决策树

Postpruning

- 错误率降低修剪 (reduced error pruning)
- 悲观错误剪枝 (pessimistic error pruning)
- 代价 - 复杂度剪枝 (cost complexity pruning)
-

- 错误率降低修剪

1. 首先删除以此结点为父结点的子树，将其作为叶子结点；
2. 将与该结点相关的数据集中的实例中最常见的类别赋予此结点；
3. 使用验证集(validation set)来分别计算此部分子树在剪枝前与剪枝后的误差；
4. 如果被叶子替代后形成的新树得到的误差等于或小于剪枝前的误差，则要对其进行剪枝，使用叶子结点替换子树；



- 悲观错误剪枝

- 令一个结点 t 的预测误差为 $e(t)$, 增大其预测误差得 $e'(t)$

$$e'(t) = e(t) + 0.5$$

- 令一个子树为 T , 其预测误差为 $e(T)$, 增大其预测误差的 $e'(T)$:

$$e'(T) = e(T) + N \times 0.5$$

其中 N 为子树 T 中叶子结点的个数; 误差修正因子0.5。

- Example: 对于一个训练数据集, 若其中有1000个实例, 总共有30个叶子结点, 最终在训练数据集上的预测错误的实例个数为10个, 设其根结点为 T , 则:

$$e(T) = 10/1000 = 1\%$$

$$e'(T) = \frac{10 + 30 \times 0.5}{1000} = 2.5\%$$

• 代价-复杂度剪枝

- 设决策树为 T , 其叶子结点的个数为 $|T|$, t 是树 T 的叶子结点, 该叶结点的样本点个数为 N_t , 属于第 k 类的样本点有 N_{tk} 个, $k=1, 2, \dots, K$, $H_t(T)$ 为叶结点 t 上的经验熵, $\alpha \geq 0$ 为参数, 此决策树学习的损失函数为

$$C_a(T) = \sum_{t=1}^{|T|} N_t H_t(T) + \alpha |T|$$

$$H_t(T) = - \sum_k \frac{N_{tk}}{N_t} \log \frac{N_{tk}}{N_t}$$

- 将损失函数的等号右边第1项记作

$$C(T) = \sum_{t=1}^{|T|} N_t H_t(T) = - \sum_{t=1}^{|T|} \sum_{k=1}^K N_{tk} \log \frac{N_{tk}}{N_t}$$

- 损失函数为

$$C_a(T) = C(T) + \alpha |T|$$

$C(T)$ 为决策树 T 对于训练数据上的预测误差。

$|T|$ 为模型的复杂程度，如果决策树 T 越复杂，则其叶子结点的个数越多，反之，决策树 T 越简单，其叶子的个数结点越少。

参数 α 控制着预测误差 $C(T)$ 与模型复杂度 $|T|$ 之间的关系。 α 越大，倾向于选择较简单的决策树模型， α 越小，倾向于选择较复杂的决策树模型。

当 α 的值确定以后，子树越大，对训练数据拟合的越好，但此时模型的复杂度也随着相应的变大；若子树越小，虽然对训练数据拟合的不好，但是此时模型的复杂程度降低了。



图 4.5 基于表 4.2 生成的未剪枝决策树

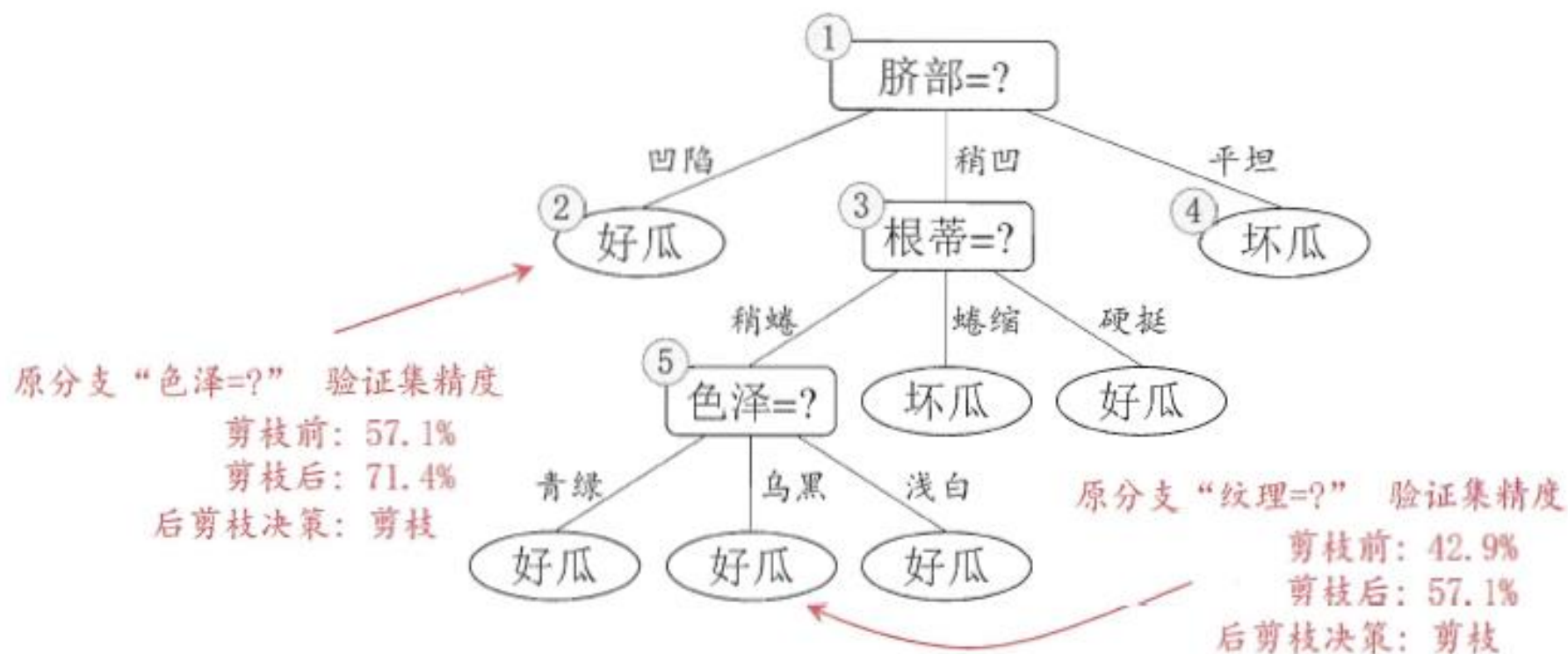


图 4.7 基于表 4.2 生成的后剪枝决策树

Decision Tree

RANDOM FORESTS

- 随机森林(random forests)使用训练数据集建立多个决策树从而组成森林。
- 新的输入实例时, 使森林中的每个决策树都进行投票
- 对于新实例的最终分类输出结果为森林中所有决策树输出最多的类别。

homework

- 5.1 根据表 5.1 所给的训练数据集，利用信息增益比（C4.5 算法）生成决策树。
- 5.2 已知如表 5.2 所示的训练数据，试用平方误差损失准则生成一个二叉回归树。

表 5.2 训练数据表

x_i	1	2	3	4	5	6	7	8	9	10
y_i	4.50	4.75	4.91	5.34	5.80	7.05	7.90	8.23	8.70	9.00

表 5.1 贷款申请样本数据表

ID	年龄	有工作	有自己的房子	信贷情况	类别
1	青年	否	否	一般	否
2	青年	否	否	好	否
3	青年	是	否	好	是
4	青年	是	是	一般	是
5	青年	否	否	一般	否
6	中年	否	否	一般	否
7	中年	否	否	好	否
8	中年	是	是	好	是
9	中年	否	是	非常好	是
10	中年	否	是	非常好	是
11	老年	否	是	非常好	是
12	老年	否	是	好	是
13	老年	是	否	好	是
14	老年	是	否	非常好	是
15	老年	否	否	一般	否

Thank You! 😊